CoffeeBeanAI: AI-Based Coffee Bean Quality Assessment

A Project Report submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Technology

in

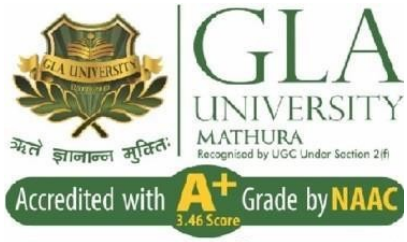
Computer Science and Engineering
Specialization in
Artificial Intelligence and Machine Learning

By
Nikhil (2415500311)

Group No. 193

Under the Guidance of
Ms. Anushka

Department of Computer Engineering & Applications
Institute of Engineering & Technology



12-B Status from UGC
GLA University
Mathura- 281406, INDIA

23rd October, 2026

DECLARATION

I hereby declare that the work presented in the B.Tech. Project “CoffeeBeanAI: AI-Based Coffee Bean Quality Assessment”, in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering with Specialization in Artificial Intelligence and Machine Learning, and submitted to the Department of Computer Engineering and Applications of GLA University, Mathura, is a record of the project work carried out by me under the supervision of Ms. Anushka.

Sign: _____
Name of Candidate: Nikhil
Univ Roll No.: 2415500311
Course: B.Tech CSE (AIML)

Sign: _____
Name of Candidate: Radhika Singh
Univ Roll No.: 2415500375
Course: B.Tech CSE (AIML)

Sign: _____
Name of Candidate: Ram Baghel
Univ Roll No.: 2415500381
Course: B.Tech CSE (AIML)

Mentor Name: Ms. Anushka

CERTIFICATE

This is to certify that the project report entitled “CoffeeBeanAI: AI-Based Coffee Bean Quality Assessment,” submitted by Nikhil, student of B.Tech. Computer Science and Engineering (Artificial Intelligence and Machine Learning), GLA University, Mathura, is a record of project work carried out under the guidance and supervision of the undersigned.

The report is submitted in partial fulfilment of the requirements for the award of the Bachelor of Technology degree.

Project Guide:
Ms. Anushka
Technical Trainer, Dept. of Computer Engg. & App.

Project Coordinator:
Mr. Preshit Mangesh Desai
Technical Trainer, Dept. of Computer Engg. & App.

Program Coordinator:
Dr. Saurabh Tiwari
Associate Professor, Dept. of Computer Engg. & App.

Date: 23rd October 2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who supported me during the work on this project, “CoffeeBeanAI: AI-Based Coffee Bean Quality Assessment”. I am thankful to my project guide, Ms. Anushka, Technical Trainer, Dept. of Computer Engg. & App., for the guidance and feedback provided during the development of this work.

I thank the Department of Computer Engineering and Applications, the Institute of Engineering & Technology, and GLA University, Mathura, for providing the academic environment and learning resources that made this project possible. I am also grateful to the Project Coordinator, Mr. Preshit Mangesh Desai, and the Program Coordinator, Dr. Saurabh Tiwari, for their support during the project.

This project relies on open-source software and publicly documented research. I gratefully acknowledge the developers and maintainers of Ultralytics YOLO, PyTorch, Hugging Face Transformers, NumPy and ONNX, as well as the authors of the DINOv2 model, whose work made it possible to build and study the pipeline described in this report. I also acknowledge the creators of the coffee-bean image dataset used in the project.

Finally, I thank my family, friends and classmates for their encouragement.

Nikhil — 2415500311
Radhika Singh — 2415500375
Ram Baghel — 2415500381

GLA UNIVERSITY, MATHURA

ABSTRACT

Visual inspection is a central step in judging the quality of coffee beans, because physical defects such as cut, immature, husk-covered, parchment-covered, partially black and shell-shaped beans reduce the uniformity of a batch. Manual sorting is slow, tiring and subjective, and its outcome can differ between inspectors. This report presents CoffeeBeanAI (the project repository is named BeanVisionAI), a computer-vision and machine-learning project that investigates a modular pipeline for locating individual coffee beans in images and classifying them into seven categories: cut, good, husk, immature, parchment, partial-black and shell.

The pipeline begins with a YOLOv8n instance-segmentation model, which locates every bean and predicts a pixel-level mask for it. Each bean is isolated using its mask and bounding box. A pretrained DINOv2 vision transformer (facebook/dinov2-base), used as a frozen feature extractor, converts each bean image into a 768-dimensional embedding, and a multilayer perceptron, CoffeeMLP (768 → 512 → 256 → 7, with batch normalisation, ReLU activations and dropout), maps the embedding to one of the seven classes. Separating segmentation from classification allows each stage to be trained, evaluated and replaced independently.

The two learned stages are evaluated separately. On a held-out test set of 215 precomputed DINOv2 feature vectors, the saved CoffeeMLP checkpoint reached an accuracy of 0.8977 (approximately 89.77%), with macro-averaged precision, recall and F1-score of 0.89, 0.90 and 0.89. This result describes the classifier operating on stored features; it is not the accuracy of the complete image-to-prediction pipeline, which has not yet been benchmarked. Segmentation metrics recorded in earlier experiments (precision 97.59%, recall 89.27%, mAP@50 94.87%, mAP@50–95 92.62%) measure a different task and are reported separately, pending confirmation against the original evaluation logs. An investigation of ONNX export and edge deployment is described as an exploratory step and not as a completed deployment.

CoffeeBeanAI is a research prototype. It has not been certified for industrial grading or validated under factory conditions. Its limitations—small and unevenly distributed test classes, dependence on mask quality, limited variation in imaging conditions, and the absence of end-to-end and latency measurements—are discussed together with directions for future work.

Keywords: coffee bean quality assessment, defect classification, instance segmentation, YOLOv8, DINOv2, feature extraction, multilayer perceptron, computer vision, deep learning, ONNX.

TABLE OF CONTENTS

DECLARATION ..... **ii**
CERTIFICATE ..... **iii**
ACKNOWLEDGEMENT ..... **iv**
ABSTRACT ..... **v**
TABLE OF CONTENTS ..... **vi**
LIST OF TABLES ..... **viii**
LIST OF FIGURES ..... **ix**
CHAPTER 1: INTRODUCTION **1**
1.1 Overview and Motivation 1
1.2 Objectives 3
1.3 Summary of Similar Approaches 5
1.4 Organization of the Project Report 5
CHAPTER 2: SOFTWARE REQUIREMENT ANALYSIS **7**
2.1 Introduction to Requirement Analysis 7
2.2 Feasibility Study 7
2.3 Functional Requirements 8
2.4 Non-Functional Requirements 9
2.5 Hardware and Software Requirements 10
2.6 Use Case Descriptions 11
2.7 Module Descriptions 12
CHAPTER 3: SOFTWARE DESIGN **13**
3.1 System Architecture Overview 13
3.2 Data Flow Diagrams 14
3.3 UML Diagrams 16
3.4 Dataset Design 17
3.5 Model Architecture 19
3.6 Preprocessing Design 21
3.7 Storage and File Organization 22
CHAPTER 4: IMPLEMENTATION AND USER INTERFACE **23**
4.1 Technology Stack Used 23
4.2 Dataset Preparation 23
4.3 YOLOv8n Segmentation Training and Inference 23
4.4 Bean Extraction 25
4.5 DINOv2 Feature Extraction 25
4.6 CoffeeMLP Classifier 26
4.7 Integration 27
4.8 Deployment Investigation 28
4.9 User Interface 29
CHAPTER 5: SOFTWARE TESTING AND RESULTS **31**
5.1 Testing Methodology 31
5.2 Segmentation Evaluation 31
5.3 Classification Evaluation 32
5.4 Confusion Matrices 33
5.5 Three Levels of Evaluation 34
5.6 Test Case Specifications 34

5.7 Error Analysis....35
5.8 Performance and Deployment Testing....36
5.9 Results Discussion....36
CHAPTER 6: CONCLUSION AND FUTURE WORK....38
6.1 Current Status and Conclusion....38
6.2 Limitations....38
6.3 Future Work....39
CHAPTER 7: SUMMARY....40
7.1 Summary....40
REFERENCES....41
APPENDIX A: LIST OF ABBREVIATIONS....42
APPENDIX B: CLASS MAPPING AND SAMPLE INFERENCE WORKFLOW....43
B.1 Class Mapping....43
B.2 Selected Configuration....43
B.3 Condensed Inference Workflow....43

LIST OF TABLES

Table 1.1 Status of the project objectives....4
Table 1.2 Comparison of approaches to coffee-bean quality assessment....5
Table 2.1 Feasibility study summary....7
Table 2.2 Functional requirements of CoffeeBeanAI....8
Table 2.3 Non-functional requirements of CoffeeBeanAI....9
Table 2.4 Software requirements and logged versions....10
Table 2.5 Hardware requirements and hardware used....10
Table 2.6 Use case: segment bean instances....11
Table 2.7 Use case: classify an extracted bean....11
Table 2.8 Use case: evaluate predictions....11
Table 3.1 Pipeline stages, inputs and outputs....13
Table 3.2 Dataset class mapping....17
Table 3.3 Recorded dataset statistics by split....18
Table 3.4 Class distribution of the 215-sample test partition....18
Table 3.5 CoffeeMLP architecture....21
Table 3.6 Preprocessing steps and verification status....21
Table 3.7 Recorded paths and files....22
Table 4.1 Technology stack summary....23
Table 4.2 YOLOv8n segmentation training configurations in the project records....24
Table 4.3 DINOv2 feature extraction configuration....26
Table 4.4 CoffeeMLP training configuration....26
Table 4.5 Deployment investigation status....28
Table 5.1 Testing levels applied to CoffeeBeanAI....31
Table 5.2 Previously reported YOLOv8n segmentation metrics....32
Table 5.3 Setup of the classifier evaluation....32
Table 5.4 Overall classification metrics on the test set....33
Table 5.5 Per-class classification metrics on the test set....33
Table 5.6 Levels of evaluation and their status....34
Table 5.7 Test case specifications and status....34
Table 5.8 Incidental timings recorded in notebook logs (not a benchmark)....36
Table 6.1 Known limitations and proposed improvements....38
Table A.1 List of abbreviations....42
Table B.1 Class mapping used throughout the project....43

LIST OF FIGURES

Figure 1.1 High-level pipeline of CoffeeBeanAI....2
Figure 1.2 Representative images of the seven classes....3
Figure 3.1 Detailed system architecture of CoffeeBeanAI....13
Figure 3.2 Conceptual project workflow of BeanVisionAI. Elements such as hyper-parameter tuning, fine-tuning loops and real-time edge inference are shown as part of the intended workflow; the status of each is stated in Chapters 4 and 5....14
Figure 3.3 Data flow diagrams (Level 0 context diagram and Level 1 detailed data flow)....15
Figure 3.4 Training data flow. The test partition is used only for the final evaluation, and labels stay aligned with their feature vectors....15
Figure 3.5 Inference data flow....16
Figure 3.6 Use case diagram....16
Figure 3.7 Dataset directory structure (CoffeeBeanMasking.yolov8)....19
Figure 3.8 CoffeeMLP architecture (reconstructed from the model definition)....21
Figure 4.1 YOLOv8n-seg output on a sample image, showing the class label and confidence predicted by the segmentation model itself....25
Figure 4.2 Per-bean labels for one input image: (a) pipeline output with CoffeeMLP labels; (b) class labels predicted directly by YOLOv8n-seg....28
Figure 4.3 Notebook inference workflow (a): loading the models and supplying an input image....29
Figure 4.4 Notebook inference workflow (b): segmentation of the input image (13 beans detected)....29
Figure 4.5 Notebook inference workflow (c): annotated result produced by the deployment notebook....30

CHAPTER 1: INTRODUCTION

1.1 Overview and Motivation

1.1.1 Background

Coffee is one of the most widely consumed beverages in the world, and the quality of the final cup depends partly on the quality of the beans that enter the roasting process. Before roasting, beans are inspected for visible defects. Typical defect and quality categories include beans that have been cut or damaged during processing, beans that are immature or poorly developed, beans that still carry fragments of husk or parchment, beans with partial black discolouration, and malformed shell-shaped beans, in addition to sound beans that show no visible defect. The seven categories used in this project are **cut, good, husk, immature, parchment, partial-black** and **shell**.

These categories are defined by appearance: colour, surface texture, shape, size and the presence of foreign layers. A trained inspector can recognise many of them at a glance, but the task becomes demanding when thousands of beans must be examined and when several categories look alike. For example, a bean with a small dark patch and a bean with a larger dark area can be difficult to separate consistently, and a bean carrying a thin layer of parchment may resemble a pale or immature bean.

1.1.2 Limitations of Manual Inspection

Manual visual inspection is flexible, but it has well-known practical limitations. It is time-consuming and its throughput is limited by the inspector's attention. Fatigue and lighting can influence decisions, and different inspectors may interpret borderline cases differently. Manual decisions are also rarely recorded in a form that can be audited later, so it is hard to measure how consistent a sorting decision was. These limitations motivate the investigation of computer-assisted visual assessment, in which a trained model supports or standardises the first-level judgement.

1.1.3 Computer Vision and Deep Learning

Computer vision allows a machine to analyse images and recognise visual patterns. Deep neural networks have become the dominant approach for this task because they can learn discriminative visual features directly from labelled examples. For coffee beans, two kinds of capability are relevant. The first is *localisation*: finding each bean in an image that may contain many beans on a background. The second is *recognition*: deciding which category an individual bean belongs to. CoffeeBeanAI treats these as two separate stages.

1.1.4 Why Separate Segmentation from Classification

An image of coffee beans usually contains several beans, and the beans may touch one another or lie at different orientations on a background. If a single model were asked to label the whole image, it would have to decide which bean each label refers to. Instance segmentation addresses this by producing, for every bean, a separate pixel-level mask. The mask is then used to extract an image of that single bean, so that the later classifier receives one bean at a time and is less influenced by neighbouring beans and by the background.

Separating the stages has further advantages. The classifier can use a pretrained feature extractor, which is useful when the labelled dataset is limited and the classes are visually similar. Each stage can be evaluated on its own, which makes errors easier to diagnose: a poor final result can be traced to a segmentation problem, a feature problem or a classification problem. The stages can also be

replaced independently, for example by testing another feature extractor without retraining the segmentation model. The price of this modularity is that errors can propagate: an inaccurate mask produces an inaccurate bean image, which can in turn produce a wrong class. This trade-off is discussed throughout the report.

1.1.5 Problem Statement

The problem addressed in this project is to design and study a pipeline that, given an image containing coffee beans, (i) identifies each bean as a separate instance, (ii) extracts the visible region of each bean, and (iii) assigns each extracted bean to one of seven visible quality or defect categories, while keeping the evaluation of the segmentation stage separate from the evaluation of the classification stage.

Why this project?

- Visual coffee-bean assessment is subjective and difficult to repeat consistently, so a measurable, repeatable software baseline is useful for study.
- A modular pipeline makes the contribution of each model component visible and testable.
- Pretrained self-supervised visual features may reduce the amount of labelled data needed for the classification stage.
- Open-source tools and notebook-based experimentation make it feasible to study such a pipeline within the resources of a student project.
- Investigating export to an interchange format (ONNX) and edge hardware gives an early view of what would be needed beyond a notebook prototype.

The project does not claim to replace expert inspectors or to guarantee accurate commercial grading. Its contribution is a documented, modular prototype and an honest account of how it is evaluated. The project is named CoffeeBeanAI in this report; the corresponding source repository is named BeanVisionAI (https://github.com/Nikhil-cloudexpert/BeanVision_AI).

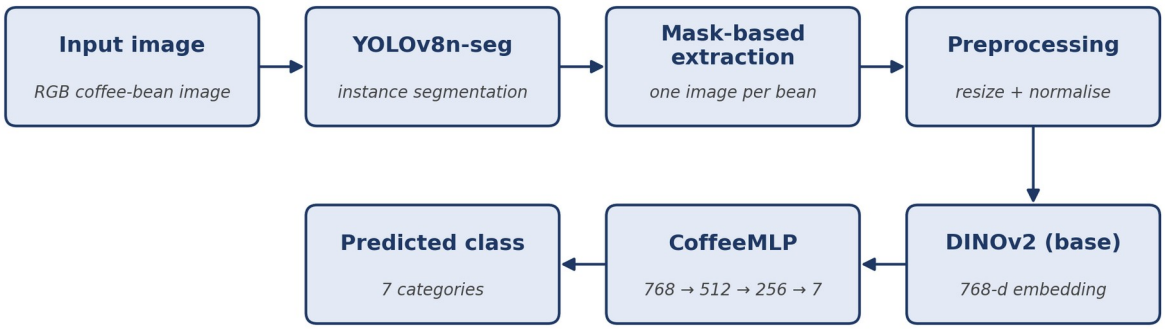


Figure 1.1: *High-level pipeline of CoffeeBeanAI*

Figure 1.1 summarises the pipeline. An input image is processed by a YOLOv8n instance-segmentation model; each detected bean is extracted and preprocessed; a DINOv2 encoder converts the bean image into a 768-dimensional embedding; and the CoffeeMLP classifier predicts one of seven classes.



Figure 1.2: *Representative images of the seven classes*

1.2 Objectives

1.2.1 Aim

The aim of the project is to investigate a modular computer-vision pipeline for coffee-bean quality assessment that combines YOLOv8n instance segmentation, DINOv2 feature extraction and an MLP classifier.

1.2.2 Objectives

1. To study the visual categories of coffee-bean quality and organise a dataset of beans into training, validation and test partitions with a consistent class mapping.
2. To train and evaluate a YOLOv8n segmentation model that identifies individual beans and produces masks.
3. To extract individual bean images using the predicted masks and prepare them for feature extraction.
4. To generate 768-dimensional visual embeddings with a pretrained DINOv2 model and store them together with correctly aligned labels.
5. To design, train and evaluate an MLP classifier (CoffeeMLP) with the architecture 768 → 512 → 256 → 7 for the seven target classes.
6. To evaluate segmentation and classification separately with appropriate metrics, and to document the limits of what each evaluation establishes.
7. To investigate ONNX export and the feasibility of running parts of the pipeline on an edge platform, without assuming that deployment succeeded.

The status of each objective at the time of writing is summarised in Table 1.1. Objectives are classed as completed, partially completed or proposed, according to the evidence described in Chapters 4 and 5.

Table 1.1: Status of the project objectives

No.	Objective (short form)	Status	Basis
1	Dataset organisation and class mapping	Completed	Train/validation/test folders and a seven-class mapping used consistently in all four notebooks.
2	YOLOv8n segmentation training and evaluation	Partially completed	Models trained; previously recorded metrics are not yet traceable to a logged run (Section 5.2).
3	Mask-based bean extraction	Completed for inference	Implemented in the deployment notebook; the crop recipe used to build the training feature set is not documented.
4	DINOv2 embeddings with aligned labels	Completed	Feature arrays of shape (4368, 768), (182, 768) and (215, 768) with matching label arrays.
5	CoffeeMLP design, training and evaluation	Completed	Checkpoint saved; test accuracy 0.8977 on 215 samples.
6	Separate evaluation of the stages	Partially completed	Segmentation (recorded) and classification (measured) evaluated separately; end-to-end evaluation not yet done.
7	ONNX export and edge feasibility	Partially completed	ONNX export recorded; no on-device execution or measurement.

1.2.3 Scope and Boundaries

The project covers image-based, single-image analysis of coffee beans in the seven classes listed above. It covers model training and evaluation in a notebook-based environment, and an initial investigation of deployment. It does not cover physical sorting hardware, conveyor or feeding mechanisms, real-time factory integration, grading according to commercial or regulatory standards, sensor fusion, or validation under industrial lighting and throughput conditions. The system has not been certified for industrial grading.

1.2.4 Expected Contribution

The expected contribution is a documented and reproducible experimental pipeline: a description of how segmentation, feature extraction and classification are connected, what data pass between them, how each stage is evaluated, and what remains to be verified. The contribution is the study of this combination of existing, publicly available tools on the project's dataset; it is not the invention of a new network architecture.

1.3 Summary of Similar Approaches

Several general approaches can be used to assess coffee beans from images. The next table compares them with the proposed pipeline. The comparison is conceptual: it describes categories of approach and their potential strengths and weaknesses rather than specific products or published performance figures.

Table 1.2: Comparison of approaches to coffee-bean quality assessment

Approach	Description	Potential strengths	Potential limitations
Manual visual inspection	Trained inspectors examine and sort beans by eye.	Flexible; no equipment beyond lighting; can use expert judgement.	Slow, fatigue-prone, subjective; decisions are hard to record and audit.
Rule-based image processing	Hand-designed colour, shape or size thresholds applied to images.	Simple and fast; easy to interpret.	Sensitive to lighting and background; difficult to encode subtle classes.
Single-stage image classifier	A CNN or transformer classifies a whole image or fixed patch.	Simple pipeline; one model to train.	Unclear which bean a label refers to when several beans are present.
Single-stage detector / segmenter with defect classes	One model locates beans and predicts the defect class directly.	End-to-end; efficient inference.	Fine-grained classes compete with localisation during training; less flexible for changing the classifier.
Proposed modular pipeline (CoffeeBeanAI)	YOLOv8n-seg for instances, DINOv2 features, MLP classifier.	Stages evaluated and replaced independently; pretrained features for limited data.	Errors can propagate between stages; several models must be maintained.

1.4 Organization of the Project Report

This report is organised into seven chapters, followed by references and appendices:

- **Chapter 1 – Introduction:** background, motivation, problem statement, objectives, scope and a comparison of approaches.
- **Chapter 2 – Software Requirement Analysis:** existing and proposed systems, feasibility, functional and non-functional requirements, hardware and software requirements, use cases and modules.
- **Chapter 3 – Software Design:** system architecture, data-flow design, use-case view, dataset design, model architecture, preprocessing design and file organisation.
- **Chapter 4 – Implementation and User Interface:** technology stack, dataset preparation, YOLOv8n training and inference, bean extraction, DINOv2 feature extraction, the CoffeeMLP classifier, integration, the deployment investigation and the notebook-based interface.
- **Chapter 5 – Software Testing and Results:** testing methodology, segmentation and classification results, the distinction between levels of evaluation, test cases, error analysis and interpretation.
- **Chapter 6 – Conclusion and Future Work:** what was done, its limitations and directions for future work.

- **Chapter 7 – Summary:** an overall recap of the project.
- **References and Appendices:** sources, abbreviations, the class mapping and a condensed inference workflow.

CHAPTER 2: SOFTWARE REQUIREMENT ANALYSIS

2.1 Introduction to Requirement Analysis

Software requirement analysis identifies what a system must do, how well it must do it and under which constraints. CoffeeBeanAI is a research prototype developed as a student project, so the "stakeholders" are the developers who run experiments, the project guide who reviews the work, and a possible future user who would apply the trained models to new images. The requirements in this chapter therefore describe an experimental machine-learning pipeline rather than a multi-user commercial product.

2.1.1 Existing System

In current practice, beans are inspected manually or with approaches based on hand-crafted image rules or on single-stage machine-learning models (see Table 1.2). Manual inspection depends on the inspector's attention and judgement and is difficult to standardise. Rule-based image processing depends on stable lighting and backgrounds and has difficulty with classes that differ in subtle texture or colour. Single-stage learning-based methods can be effective, but when a single model must both find each bean and distinguish similar categories, the two tasks interact, and it becomes harder to tell whether a mistake came from localisation or from classification. No claim is made here about any specific commercial system.

2.1.2 Proposed System

The proposed system is a modular pipeline. An image is passed to a YOLOv8n segmentation model that returns the beans it finds, each with a mask. The mask is used to extract the bean from the image. The extracted bean is preprocessed and passed through a pretrained DINOv2 encoder to obtain a 768-dimensional embedding. The CoffeeMLP classifier converts the embedding into scores for the seven classes, and the highest-scoring class is reported. Segmentation and classification are evaluated separately. The motivation for this separation is that individual beans are isolated before classification, pretrained features reduce the need for large labelled data, and each stage can be diagnosed and replaced independently.

2.2 Feasibility Study

A feasibility study was carried out to judge whether the project could be completed with the available resources. The findings are summarised below. No development cost figures or commercial viability claims are made.

Table 2.1: Feasibility study summary

Feasibility type	Analysis	Conclusion
Technical	All required components—Python, PyTorch, Ultralytics YOLO, Hugging Face Transformers, NumPy and ONNX—are open-source, documented and widely used. Pretrained YOLOv8n-seg and DINOv2 checkpoints are available, so training from scratch is not required. GPU access through a hosted notebook environment is sufficient for the model sizes used.	Feasible

Feasibility type	Analysis	Conclusion
Operational	The prototype is operated from notebooks by a developer. No specialised operator training is required beyond familiarity with Python notebooks. Use in a sorting workflow would require further engineering and validation.	Feasible for research use
Economic	Development relies on open-source libraries and a hosted notebook environment with cloud storage. No commercial viability analysis was performed.	Feasible within student-project resources
Scheduling	Work proceeds in stages that can be completed separately: dataset preparation, segmentation training, feature extraction, classifier training, evaluation and deployment investigation.	Feasible

Open-source libraries lower the barrier to entry and allow the experiments to be repeated by others. Notebook-based experimentation allows quick iteration and inspection of intermediate outputs, though it requires discipline to keep experiments reproducible. The licence terms of each library and pretrained checkpoint should be reviewed before any redistribution or commercial use.

2.3 Functional Requirements

Functional requirements define what the system must do. The next table lists them. The Status column reflects what is present in the project notebooks: *Implemented* means the behaviour is present in the notebook code, *Investigated* means it was explored but not completed, and *Proposed* means it is specified here but not present in the notebooks.

Table 2.2: Functional requirements of CoffeeBeanAI

ID	Requirement	Description and expected behaviour	Status
FR-01	Image loading	Load RGB images from storage or from an upload cell.	Implemented
FR-02	Dataset and annotation handling	Read the train/validation/test organisation, segmentation labels and dataset configuration; check that class indices are valid.	Implemented (via Ultralytics dataset loading)
FR-03	Bean instance segmentation	Run the YOLOv8n segmentation model and return, for each bean, a bounding box, a mask and a confidence score.	Implemented
FR-04	Mask-based bean extraction	Use each predicted mask and box to produce one image per bean.	Implemented
FR-05	Image preprocessing	Resize and normalise each bean image with the DINOv2 image processor so that the input format is consistent.	Implemented
FR-06	DINOv2 feature generation	Produce one 768-dimensional embedding per bean image and save train, validation and test feature arrays with aligned labels.	Implemented
FR-07	MLP classification	Map each embedding to seven class scores with CoffeeMLP (768 → 512 → 256 → 7).	Implemented
FR-08	Class-index mapping	Convert predicted indices 0–6 into the labels cut, good, husk, immature, parchment, partial-black and shell.	Implemented

ID	Requirement	Description and expected behaviour	Status
FR-09	Model evaluation	Compute segmentation metrics (precision, recall, mAP) and classification metrics (accuracy, precision, recall, F1, confusion matrix) separately.	Classification implemented; segmentation metrics recorded
FR-10	Result storage or display	Display or save predictions and evaluation outputs for later inspection.	Implemented (notebook display; annotated image saved)
FR-11	Error handling	Handle missing files, images with no detected bean, and mismatched array shapes with informative messages.	Proposed (not present in the notebooks)
FR-12	Checkpoint loading	Load the trained segmentation checkpoint and the classifier weights.	Implemented
FR-13	Model export	Export the trained segmentation model to ONNX for deployment experiments.	Investigated

2.4 Non-Functional Requirements

Non-functional requirements describe the qualities expected of the prototype. They are deliberately modest, reflecting a research prototype.

Table 2.3: Non-functional requirements of CoffeeBeanAI

Quality	Requirement
Reliability	The pipeline should handle absent detections and unreadable inputs without crashing, and should not silently produce labels when an earlier stage failed.
Maintainability	Each stage (segmentation, extraction, feature extraction, classification, evaluation) should be separable code so that one stage can be modified without rewriting others.
Reproducibility	Library versions, checkpoint identities, dataset partitions, random seeds and key hyper-parameters should be recorded so that experiments can be repeated.
Performance	Inference should complete in a time suitable for offline and interactive experimentation. Latency and throughput must be measured before any real-time claim is made.
Usability	Inputs, outputs and class labels should be easy to inspect in the notebook; outputs should show the predicted label for every bean.
Extensibility	It should be possible to replace the feature extractor or classifier, or to add classes, with limited changes.
Resource efficiency	Models should be small enough for hosted notebooks; feature extraction should be batched; later edge deployment should be assessed with measured memory use.
Evaluation integrity	The test partition must be used only for final evaluation, no image may appear in more than one partition, and segmentation metrics must not be reported as classification performance.

2.5 Hardware and Software Requirements

The software stack of the project consists of Python, PyTorch, the Ultralytics library, Hugging Face Transformers, NumPy and a notebook-based environment. Google Colab and Google Drive formed part of the workflow. The software versions below are those that appear in the logged notebook sessions; versions that were not logged are stated as such.

Table 2.4: Software requirements and logged versions

Software	Version	Role
Python	3.12.13 (Colab session log)	Programming language for all stages.
PyTorch	2.11.0 (+cu128) (Colab session log)	Tensor computation and neural-network training for CoffeeMLP and DINOv2 inference.
Ultralytics	8.4.70 (training notebook log)	YOLOv8n segmentation training, validation, inference and export.
Hugging Face Transformers	Not recorded in the supplied logs	Loading the DINOv2 image processor and model (facebook/dinov2-base).
NumPy	2.0.2 (Colab session log)	Storage and manipulation of feature and label arrays.
ONNX / ONNX Runtime	Not recorded	Model export and runtime-loading experiments.
Google Colab and Google Drive	—	Notebook environment and storage for datasets, features and checkpoints.

The hardware column below separates what is indicated for development from what the notebook logs show was used. The Arduino UNO Q is the platform targeted in the deployment investigation (Section 4.8); it has not yet been used to run the pipeline.

Table 2.5: Hardware requirements and hardware used

Item	Indicative minimum for development	Used in the recorded notebooks
Processor	Multi-core CPU.	Colab CPU runtime for the deployment notebook (device reported as cpu).
GPU	A CUDA-capable GPU is recommended for training; CPU-only is possible for small inference runs.	NVIDIA Tesla T4 (about 15 GB memory) in the segmentation, feature-extraction and classifier notebooks.
RAM	8 GB or more recommended.	System RAM of the Colab sessions was not logged.
Storage	Sufficient for the dataset, feature arrays and checkpoints.	Google Drive.
Edge platform	Not required for development.	Arduino UNO Q (Qualcomm Dragonwing QRB2210 microprocessor, 2 GB or 4 GB RAM depending on variant [17]) targeted; not yet used for inference.

2.6 Use Case Descriptions

The primary actor is the researcher or developer who runs the notebooks. The following use cases describe the main interactions. A use-case diagram is given in Section 3.3.

Table 2.6: Use case: segment bean instances

Field	Detail
Use case ID / name	UC-01 / Segment bean instances
Actor	Researcher / developer
Preconditions	A trained YOLOv8n segmentation checkpoint is available; the input image exists and is readable.
Main flow	1. The actor supplies an image. 2. The system loads the checkpoint. 3. The model predicts instances. 4. The system returns boxes, masks and confidence scores for each bean.
Expected output	A set of bean instances with masks (possibly empty).
Exceptions	Missing file or checkpoint: an error message is shown. No bean detected: an empty result is returned and reported.

Table 2.7: Use case: classify an extracted bean

Field	Detail
Use case ID / name	UC-02 / Classify an extracted bean
Actor	Researcher / developer
Preconditions	One or more bean images are available; the DINOv2 model and CoffeeMLP weights can be loaded.
Main flow	1. The system preprocesses each bean image. 2. DINOv2 produces a 768-dimensional embedding. 3. CoffeeMLP produces seven class scores. 4. The highest-scoring index is mapped to a class label.
Expected output	A class label (cut, good, husk, immature, parchment, partial-black or shell) for each bean.
Exceptions	Embedding of the wrong size or incompatible weights: the system reports an error instead of predicting.

Table 2.8: Use case: evaluate predictions

Field	Detail
Use case ID / name	UC-03 / Evaluate predictions
Actor	Researcher / developer
Preconditions	Held-out validation or test data with ground-truth labels; predictions from the model under evaluation.
Main flow	1. The actor selects the stage (segmentation or classification). 2. The system computes the metrics defined for that stage. 3. Results are displayed or saved with the checkpoint and data partition identified.
Expected output	Segmentation: precision, recall, mAP@0.50, mAP@0.50:0.95. Classification: accuracy, per-class precision, recall, F1-score, confusion matrix.

Field	Detail
Exceptions	Label–prediction count mismatch or class-index inconsistency: evaluation is stopped and the inconsistency reported.

2.7 Module Descriptions

The system is organised into six modules, each corresponding to a stage of the workflow.

- **Module 1: Dataset and Annotation Module.** Handles the organisation of images and labels into training, validation and test partitions, the dataset configuration, and the class mapping.
- **Module 2: Segmentation Module.** Trains and runs the YOLOv8n instance-segmentation model and provides boxes, masks and confidence scores.
- **Module 3: Bean Extraction and Preprocessing Module.** Converts masks and boxes into individual bean images and prepares them for the feature extractor.
- **Module 4: Feature Extraction Module.** Uses the pretrained DINOv2 encoder to produce 768-dimensional embeddings and stores feature arrays with aligned labels.
- **Module 5: Classification Module.** Contains the CoffeeMLP classifier, its training and inference, and the mapping from output indices to labels.
- **Module 6: Evaluation and Deployment Investigation Module.** Computes metrics for each stage and supports the ONNX export and edge-deployment experiments.

CHAPTER 3: SOFTWARE DESIGN

3.1 System Architecture Overview

CoffeeBeanAI is designed as a sequential, modular pipeline in which each stage consumes the output of the previous stage. The design has three learned components—a YOLOv8n-seg segmentation model, a frozen DINOv2 encoder and the CoffeeMLP classifier—connected by data-preparation steps. The architecture is shown below, and the next table lists the data passed between stages.

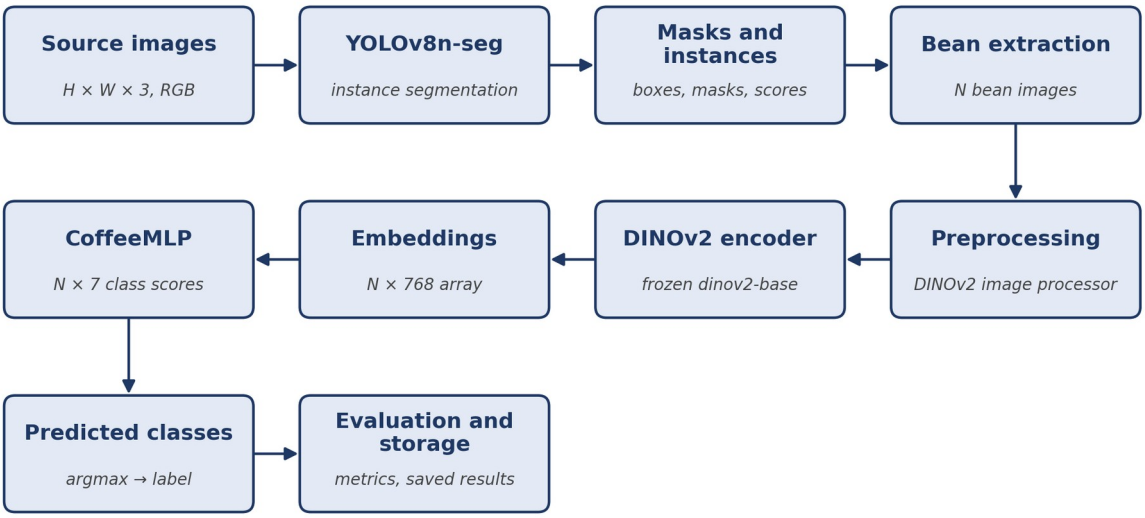


Figure 3.1: *Detailed system architecture of CoffeeBeanAI*

Table 3.1: Pipeline stages, inputs and outputs

Stage	Input	Process	Output
1. Source image	Image file	Load as RGB array.	Array of shape H × W × 3.
2. Segmentation	RGB image	YOLOv8n-seg predicts instances.	N instances: boxes, confidences, masks.
3. Bean extraction	Image and masks	Isolate each bean using its mask and box.	N bean images.
4. Preprocessing	Bean images	DINOv2 image processor resizes and normalises.	Tensor of pixel values.
5. Feature extraction	Pixel values	Forward pass through frozen DINOv2.	N × 768 embeddings.
6. Classification	Embeddings	CoffeeMLP forward pass.	N × 7 class scores.
7. Prediction	Class scores	Highest score; index-to-label mapping.	Class label per bean.
8. Evaluation / storage	Predictions and ground truth	Compute metrics; save or display results.	Metrics and stored results.

The first three stages localise and isolate the beans; stages four to seven recognise them; stage eight measures quality. Because stage 3 depends on stage 2, the quality of the masks directly limits the quality of the bean images supplied to the classifier. The overall project workflow, including the optional deployment step, is illustrated in the next figure.

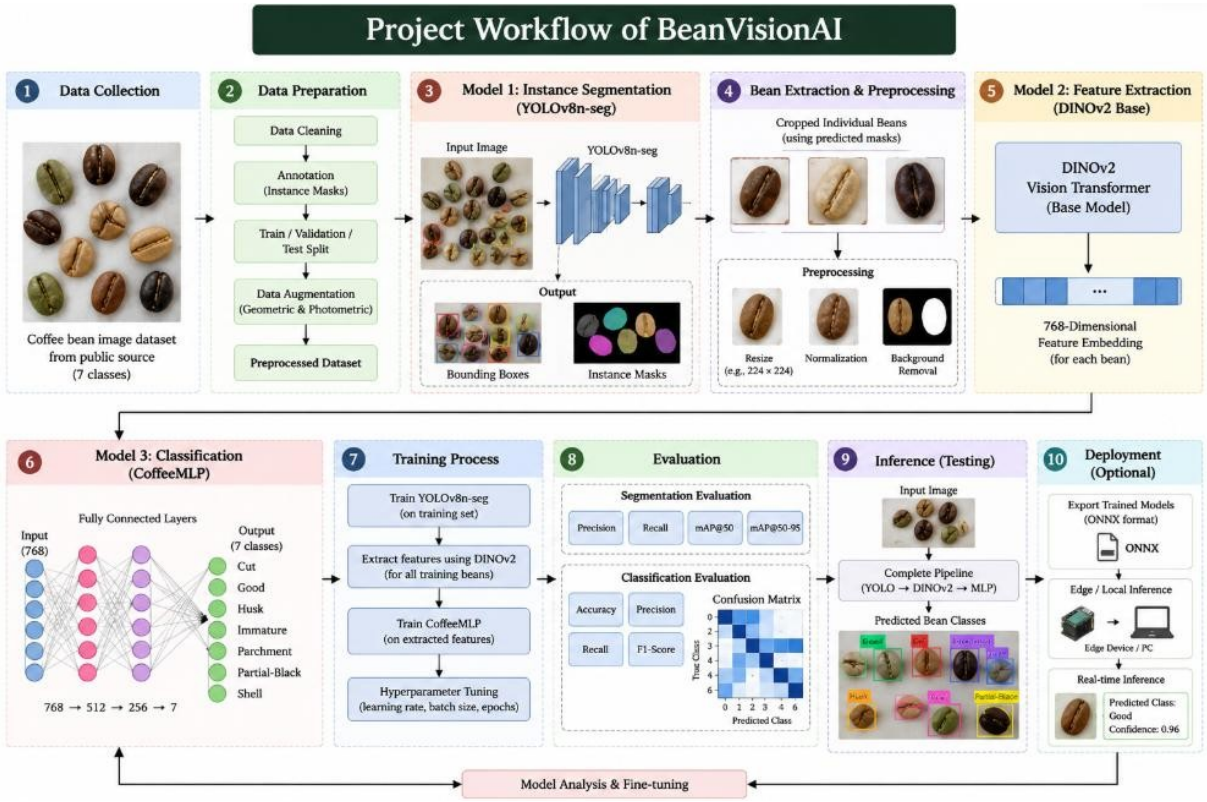


Figure 3.2: *Conceptual project workflow of BeanVisionAI. Elements such as hyper-parameter tuning, fine-tuning loops and real-time edge inference are shown as part of the intended workflow; the status of each is stated in Chapters 4 and 5*

3.2 Data Flow Diagrams

3.2.1 Level 0 – Context

At the context level, CoffeeBeanAI is a single process with one external entity, the researcher or developer. The entity supplies coffee-bean images, annotations, pretrained checkpoints and configuration. The system returns bean masks, class predictions, evaluation metrics and exported models. The dataset and checkpoint stores (Google Drive in the recorded workflow) act as data stores.

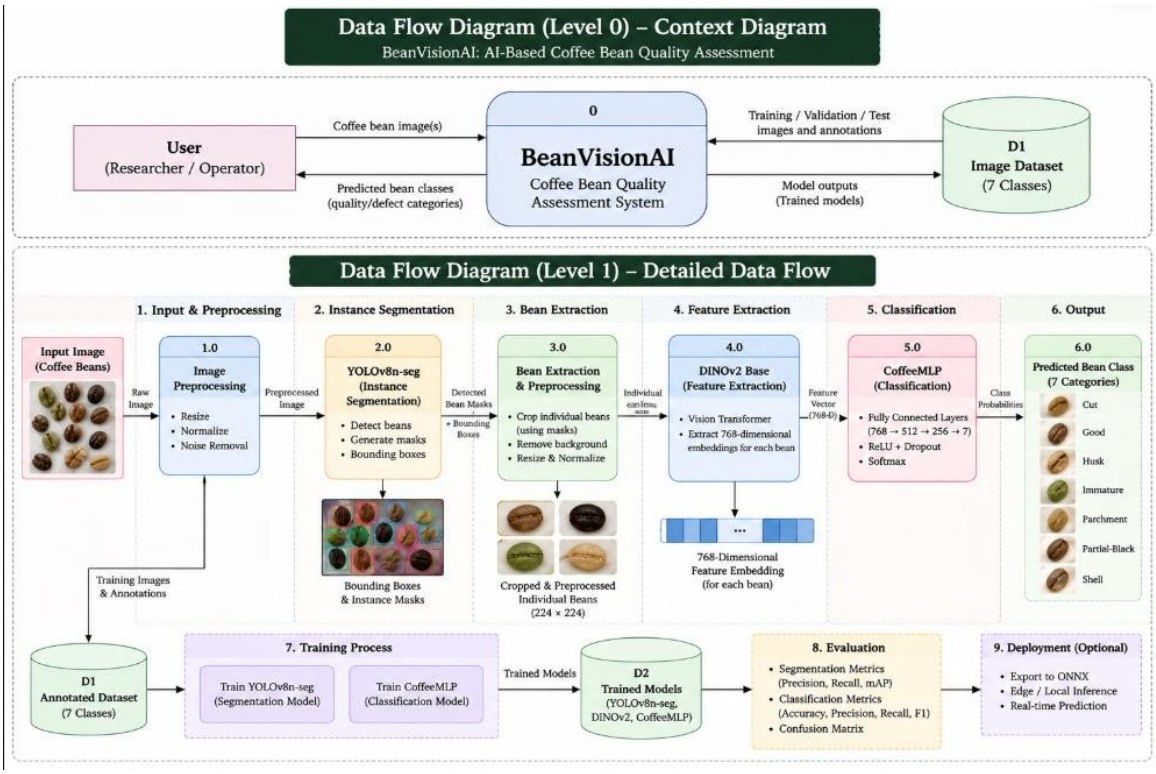


Figure 3.3: *Data flow diagrams (Level 0 context diagram and Level 1 detailed data flow)*

3.2.2 Training Workflow

Training consists of two largely independent stages. In Stage A, annotated images and segmentation labels, described by a dataset configuration, are used to train the YOLOv8n-seg model, starting from the pretrained yolov8n-seg.pt weights. Validation results guide checkpoint selection, and segmentation metrics are computed. In Stage B, bean images with class labels from the training, validation and test partitions are passed through the frozen DINOv2 encoder. The resulting 768-dimensional embeddings and their labels are saved as NumPy arrays, which are then used to train CoffeeMLP. The validation partition supports checkpoint selection, and the test partition is reserved for the final evaluation.

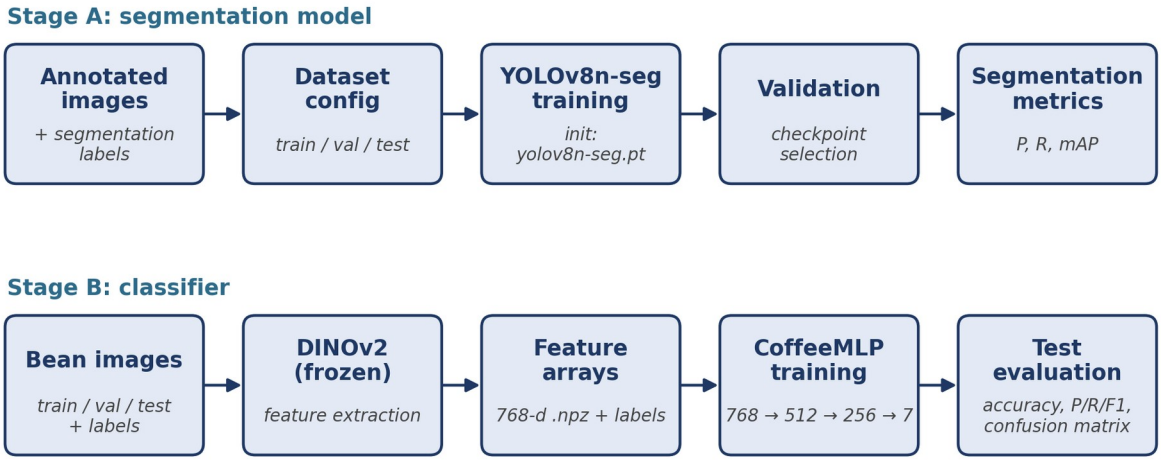


Figure 3.4: *Training data flow. The test partition is used only for the final evaluation, and labels stay aligned with their feature vectors*

3.2.3 Inference Workflow

At inference time only trained components are used. The image is loaded and the segmentation checkpoint predicts masks. Each bean is extracted and preprocessed, DINOv2 produces its embedding, CoffeeMLP produces class scores, and the index of the highest score is converted into a class name. The output is a label for every detected bean. No labels are needed at this stage.

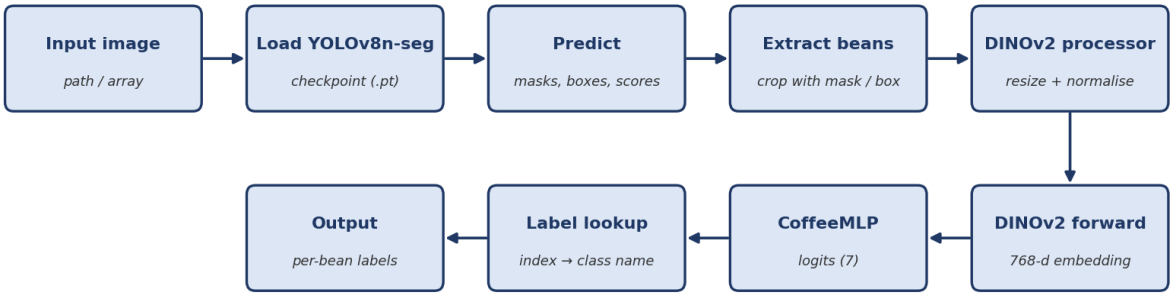


Figure 3.5: *Inference data flow*

3.3 UML Diagrams

3.3.1 Use Case Diagram

The use-case diagram shows the actor and the use cases of the prototype. The researcher loads the checkpoints, segments bean instances, extracts bean regions, generates embeddings, classifies beans and evaluates predictions.

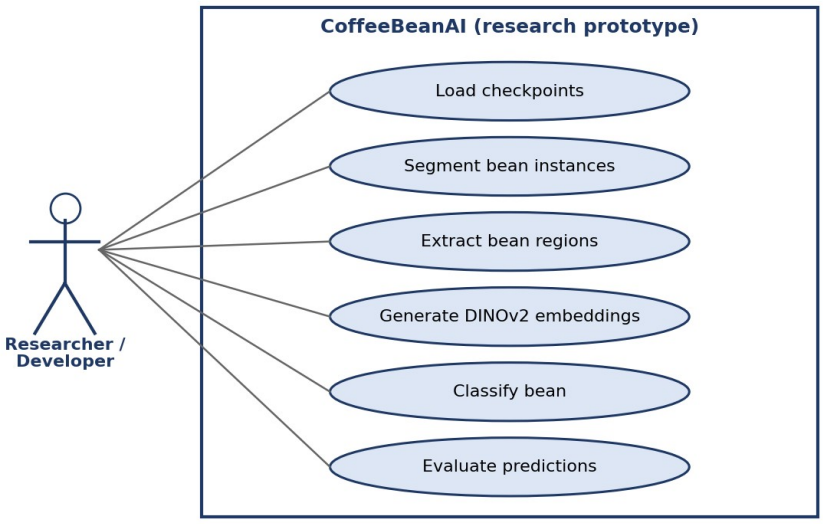


Figure 3.6: *Use case diagram*

3.3.2 Sequence of Interactions during Inference

The inference sequence is: (1) the researcher supplies an image; (2) the notebook loads the YOLOv8n-seg checkpoint and runs prediction; (3) the notebook extracts each bean using its mask; (4) the DINOv2 processor prepares the bean images; (5) the DINOv2 model returns the embeddings; (6) the CoffeeMLP returns class scores; (7) the notebook maps the highest-scoring index to a class label and displays or stores the results. Each step passes its output to the next as an

in-memory array or tensor. A class diagram is not included because the project is organised as notebook code rather than a designed object-oriented application.

3.4 Dataset Design

3.4.1 Organisation and Class Mapping

The dataset is organised into training, validation and test directories in the YOLOv8 segmentation format (CoffeeBeanMasking.yolov8). In the working environment it was stored as newMaskedDataset on Google Drive. The original source, authorship and licence of the dataset are not documented in the project material; the project does not claim ownership of the underlying images or annotations. The seven classes and their indices are given in the next table. The descriptions are general descriptions of the visual categories; the exact labelling criteria are those of the dataset's own documentation.

Table 3.2: Dataset class mapping

Class ID	Class label	General visual description
0	cut	Bean with a visible cut or mechanical damage.
1	good	Bean with no visible defect.
2	husk	Bean with fragments of dried husk attached.
3	immature	Under-developed bean, often small, pale or wrinkled.
4	parchment	Bean still covered, wholly or partly, by its parchment layer.
5	partial-black	Bean with a partly dark or black area.
6	shell	Malformed, shell-shaped bean.

Class-index consistency is essential. The same index-to-label mapping must be used in the dataset configuration, the saved label arrays, the classifier's output layer and the inference code. If a mapping differed between the training and inference code, the model would still produce outputs, but they would be assigned to the wrong names, and this error would not raise an exception. In the project notebooks the mapping is identical in the dataset configuration (data.yaml class names), the feature-extraction label dictionary, the classifier evaluation and the deployment notebook. The segmentation model was trained with these same seven classes (the Ultralytics log reports nc = 7), so it produces its own class predictions in addition to the masks; in the deployment notebook the final label of each bean is the one predicted by CoffeeMLP, and the segmentation model's class output is not used for that label (see Section 4.7).

3.4.2 Split Statistics

The recorded dataset statistics are given in the next table. Images, annotated bean instances, extracted bean images, feature vectors and test samples are different quantities and are not interchangeable. An image usually contains several beans, so there are more bean instances than images. The feature arrays contain one vector per bean instance, and the shapes recorded in the feature-extraction notebook equal the recorded instance counts. The counts are the project's recorded statistics for each split; they have not been independently re-counted for this report, and the project material does not establish that all of them refer to the same dataset revision.

Table 3.3: Recorded dataset statistics by split

Split	Images	Annotated bean instances	DINOv2 feature array shape	Label array shape
Training	2,070	4,368	(4368, 768)	(4368,)
Validation	86	182	(182, 768)	(182,)
Test	86	215	(215, 768)	(215,)
Total	**2,242**	**4,765**	—	—

In the validation run of the segmentation notebook, the Ultralytics validator counted 180 instances in the 86 validation images and reported that two duplicate labels had been removed; this is consistent with the recorded count of 182 annotated instances, but it shows that the number of instances actually used by the segmentation validator can differ slightly from the number of annotations on disk.

The per-class distribution of the test partition, taken from the classification report of the classifier evaluation, is given next. The distribution is uneven: *good* and *husk* together make up about 52% of the test samples, whereas *immature* and *parchment* contribute 11 samples each (about 5% each). Per-class counts for the training and validation partitions are not reproduced here because the per-class figures recorded in earlier drafts did not sum to the recorded split totals and could not be verified.

Table 3.4: Class distribution of the 215-sample test partition

Class	Class ID	Test samples	Share of test set
cut	0	23	10.7%
good	1	58	27.0%
husk	2	54	25.1%
immature	3	11	5.1%
parchment	4	11	5.1%
partial-black	5	31	14.4%
shell	6	27	12.6%
Total		**215**	**100.0%**

When the same distribution is considered for the whole project, class imbalance affects how results should be read: if some classes have many more examples than others, overall accuracy can hide weak performance on minority classes, which is why per-class metrics are reported in Chapter 5.

3.4.3 Quality and Leakage Considerations

Annotation quality affects both stages: inconsistent boundaries lower the quality of the segmentation labels, and inconsistent class labels limit what the classifier can learn. Data leakage must be avoided. Leakage occurs when the same bean, or near-identical images of it, appear in more than one partition, or when beans cropped from one image are placed in different partitions. The project material does not document how bean-level samples were assigned to partitions relative to their source images, so the absence of leakage is an assumption of the design rather than a verified property. The test partition should remain independent: it should not be used for choosing checkpoints or hyper-parameters. Background, lighting, bean orientation and image quality also influence results, and the dataset's variation in these factors determines how far results can be generalised.

```
CoffeeBeanMasking.yolov8/
│
├── train/
│   ├── images/   (2,070 images)
│   └── labels/   (YOLO segmentation annotations)
│
├── valid/
│   ├── images/   (86 images)
│   └── labels/   (YOLO segmentation annotations)
│
├── test/
│   ├── images/   (86 images)
│   └── labels/   (YOLO segmentation annotations)
│
└── data.yaml     (7 class definitions and dataset paths)
```

Figure 3.7: *Dataset directory structure (CoffeeBeanMasking.yolov8)*

3.5 Model Architecture

3.5.1 YOLOv8n Instance Segmentation

Object detection predicts a bounding box and a class for each object. Instance segmentation additionally predicts, for each object instance, a mask that marks which pixels belong to it. Semantic segmentation, by contrast, labels every pixel with a class but does not separate individual objects of the same class; for counting and classifying individual beans, instance-level masks are therefore the appropriate output. The bounding box gives a coarse location; the mask gives the object's shape and separates it from the background and, in many cases, from neighbouring objects. YOLO models belong to the family of single-stage detectors introduced in [3], and the instance-segmentation task is related to region-based methods such as Mask R-CNN [4]. YOLOv8 is implemented in the Ultralytics library [1], [2]. Its segmentation variant adds a mask branch to the detection head: it predicts a set of mask coefficients for each instance together with a set of prototype masks, and combines them into the instance mask. The "n" (nano) variant is the smallest model size and is

therefore suited to limited compute; the training log of the project reports 152 layers and about 3.26 million parameters for the seven-class model.

The role of YOLOv8n-seg in CoffeeBeanAI is to supply, for each bean, a mask and box from which the bean can be extracted. During training it learns from annotated images and segmentation labels; the dataset configuration lists the image directories and class names and must agree with the class indices in the label files.

3.5.2 DINOv2 Feature Extractor

DINOv2 is a family of vision transformers trained with self-supervised learning on a large curated image collection, producing general-purpose visual features without task-specific labels [5]. A vision transformer divides an image into fixed-size patches, converts each patch into a token, and processes the tokens with self-attention layers [6]. The facebook/dinov2-base model, loaded through Hugging Face Transformers [7], [8], has a hidden size of 768, which defines the dimension of the embedding used in this project.

A pretrained representation is useful when the labelled dataset is limited, because the encoder has already learned to represent edges, textures, shapes and object parts. In CoffeeBeanAI, DINOv2 is used as a frozen feature extractor: the model is put in evaluation mode, gradients are not computed, its parameters are not updated, and only the small classifier is trained on the embeddings. Fine-tuning would update the encoder's parameters as well, which can improve adaptation to the task but requires more compute and more care to avoid over-fitting. The project uses feature extraction and not fine-tuning. Because feature extraction and classifier training are separate stages, embeddings are computed once, stored, and reused for repeated classifier experiments.

3.5.3 CoffeeMLP Classifier

CoffeeMLP is a multilayer perceptron that maps one 768-dimensional embedding to seven class scores. Its architecture is 768 → 512 → 256 → 7 (see the architecture figure and table below). Each of the two hidden blocks consists, in this order, of a linear layer, batch normalisation, a ReLU activation and dropout (probability 0.30 in the first block and 0.20 in the second); the output block is a single linear layer. This layer order was read from the model definition and printed summary in the classifier and deployment notebooks. Each linear layer computes a weighted sum of its inputs:

$$z = Wx + b \tag{3.1}$$

where x is the input vector, W the weight matrix, b the bias vector and z the layer output. ReLU, defined as ReLU(z) = max(0, z), introduces non-linearity [13], without which a stack of linear layers would be equivalent to a single linear layer. Batch normalisation normalises layer activations using batch statistics, which can stabilise optimisation [11]. Dropout randomly sets a fraction of activations to zero during training, which can reduce over-fitting [12]; it is inactive in evaluation mode. The final layer has seven units, one per class; its outputs are scores (logits). A probability-like distribution can be obtained with the softmax function:

$$p_k = exp(z_k) / \Sigma_j exp(z_j), \quad k = 0, 1, \ldots, 6 \tag{3.2}$$

and the predicted class is the index k with the largest score. These softmax values are model confidence scores; they are not guaranteed to be calibrated probabilities. The classifier is trained with the cross-entropy loss and the Adam optimiser, as described in Section 4.6.

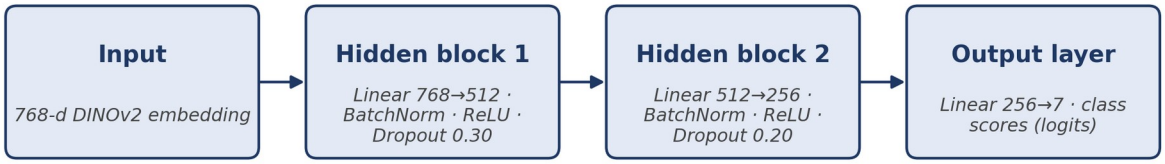


Figure 3.8: *CoffeeMLP architecture (reconstructed from the model definition)*

Table 3.5: CoffeeMLP architecture

Block	Input → output	Components (in order)	Parameters†
Hidden 1	768 → 512	Linear, batch normalisation, ReLU, dropout (p = 0.30)	393,728 (linear); 1,024 (BN)
Hidden 2	512 → 256	Linear, batch normalisation, ReLU, dropout (p = 0.20)	131,328 (linear); 512 (BN)
Output	256 → 7	Linear (class scores)	1,799
Total	**768 → 7**	**2 hidden blocks and 1 output layer**	**≈ 528,391 trainable**

† Derived arithmetically (inputs × outputs + biases; two trainable values per batch-normalisation unit) from the architecture in the notebook; they were not read from a model-summary tool.

3.6 Preprocessing Design

Preprocessing converts raw masks and images into inputs with a consistent format. The first step applies the segmentation output to the image to isolate each bean; the second prepares the resulting image for DINOv2. Consistent resizing, normalisation and channel order are essential: the DINOv2 encoder expects inputs prepared in the way the Hugging Face image processor defines, and embeddings from differently prepared images are not comparable. The same preprocessing should be used for training, validation, testing and inference. The steps are listed below with their verification status.

Table 3.6: Preprocessing steps and verification status

Step	Description	Status
Mask application	Each predicted mask is resized to the image size and thresholded at 0.5; pixels outside the mask are set to black.	Implemented in the deployment notebook
Bean extraction	The masked image is cropped to the integer bounding box of the instance, giving one image per bean.	Implemented in the deployment notebook
Resizing	The crop is resized to 224 × 224 pixels with OpenCV; the aspect ratio is not preserved.	Implemented in the deployment notebook
Colour order	OpenCV's BGR array is converted to RGB and wrapped as a PIL image.	Implemented in the deployment notebook
DINOv2 image processing	The Hugging Face AutoImageProcessor of facebook/dinov2-base is applied. Its published default configuration resizes the shorter side to 256 pixels, centre-crops to 224 × 224 and normalises with ImageNet statistics; the exact values are those in the model's preprocessor_config.json.	Implemented; settings follow the model's configuration file
Batching	Feature extraction for the training set used batches of 32 images; the deployment notebook processes beans one at a time.	Batched in feature extraction; not batched at inference

Step	Description	Status
Training-set crop recipe	The recipe used to produce the bean images from which the training features were computed is not part of the supplied notebooks.	Not documented; see Section 6.2

Because the bean image is first resized to 224 × 224 and the processor then resizes and centre-crops, the outer margin of each bean image may be discarded before feature extraction. Whether this affects elongated beans or edge defects has not been tested and is noted as a point for further investigation.

3.7 Storage and File Organization

The recorded storage locations are listed below. The Google Drive paths are historical notes from the notebook environment; the repository paths are those of the project repository. The two recorded segmentation checkpoints do not necessarily belong to the same experiment.

Table 3.7: Recorded paths and files

Path or file	Recorded purpose
/content/drive/MyDrive/newMaskedDataset/ newMaskedDataset	Dataset root (train / valid / test with images, labels and data.yaml).
/content/drive/MyDrive/DINO_Features	Saved DINOv2 feature arrays (train_data.npz, valid_data.npz, test_data.npz) and the classifier checkpoint mlp_classifier.pth.
/content/drive/MyDrive/Copy of v8_75.pt	Segmentation checkpoint loaded by the deployment notebook.
/content/runs/segment/train/weights/best.pt	Checkpoint produced by the segmentation training notebook (640-pixel run).
models/MLP/mlp_classifier.pth (repository)	Saved CoffeeMLP checkpoint evaluated in Chapter 5.
data/processed/test_data.npz (repository)	Processed test features and labels used for the classifier evaluation.
src/evaluate_mlp.py (repository)	Evaluation script for the saved classifier.
outputs/confusion_matrix/, outputs/metrics/, outputs/predictions.csv, outputs/logs/mlp_evaluation.txt (repository)	Confusion matrices, classification report, evaluation summary, predictions and evaluation log.
4_Deployment_Pipeline.ipynb	Notebook that combines segmentation, feature extraction and classification.

Credentials, tokens and other secrets must not be stored in notebooks or in this report.

CHAPTER 4: IMPLEMENTATION AND USER INTERFACE

This chapter describes how the design of Chapter 3 was implemented. Details are taken from the project notebooks and their logged outputs; where a detail is not available in those sources, this is stated.

4.1 Technology Stack Used

The implementation uses open-source libraries in a notebook environment. The next table gives the role of each tool.

Table 4.1: Technology stack summary

Layer	Technology	Purpose
Language	Python	Implementation of all stages.
Deep learning	PyTorch	Tensors, CoffeeMLP definition, training and inference [9], [10].
Segmentation	Ultralytics YOLO	YOLOv8n-seg training, evaluation, inference and ONNX export [1], [2].
Feature extraction	Hugging Face Transformers	DINOv2 image processor and model [7], [8].
Data	NumPy, scikit-learn, OpenCV, Pillow	Feature and label arrays [16]; classification metrics; image handling.
Interchange	ONNX	Exported model format [15].
Environment	Google Colab, Google Drive	Notebook execution and storage.

Library versions that appear in the logged sessions are given in Table 2.4.

4.2 Dataset Preparation

The dataset was organised into training, validation and test directories under the dataset root listed in Section 3.7. Preparation involved loading the images, reading the annotation and label files, and checking that the class mapping of Section 3.4 is the same in the dataset configuration, the saved label arrays and the inference code. The segmentation notebook also scanned the label folders for empty label files and counted labels per split; during training, the Ultralytics loader reported 12 background images in the training split and removed a small number of duplicate labels (four training images and one validation image). The classification feature set was organised in class folders (cut, good, husk, immature, parchment, partial-black, shell) for each split.

4.3 YOLOv8n Segmentation Training and Inference

The segmentation model was created from the pretrained yolov8n-seg.pt checkpoint and trained with the Ultralytics library. The sequence is: initialise the model from the pretrained weights; point the trainer to the dataset configuration; train while the library validates on the validation partition after each epoch; keep the best checkpoint according to the library's validation criterion; and evaluate.

Two training configurations appear in the project records, and they belong to different records. The first is the configuration of the segmentation training notebook, whose log is available: 100 epochs requested, batch size 16, image size 640, and automatic optimiser selection (the log shows AdamW with a learning rate of about 0.0009, so the nominal initial learning rate of 0.01 was overridden). That run was interrupted manually during epoch 8, so its checkpoints (best.pt and last.pt) come from a partially trained model. The second is the configuration recorded for the checkpoint v8_75.pt, which is the checkpoint loaded by the deployment notebook: 150 epochs, batch size 16 and an image size of 1024 × 1024 (the ONNX export of this checkpoint has a nominal input size of 1024 × 1024, whereas the export of the 640-pixel checkpoint is 640 × 640). No training log of the v8_75.pt run was available for this report, so its values are given as recorded and have not been independently confirmed.

Table 4.2: YOLOv8n segmentation training configurations in the project records

Parameter	Segmentation training notebook (log available)	Checkpoint v8_75.pt (recorded; no log)
Ultralytics version	8.4.70	8.4.64 (recorded)
Initial weights	yolov8n-seg.pt (pretrained)	yolov8n-seg.pt (recorded)
Number of classes	7 (nc = 7)	7 (data.yaml)
Epochs	100 requested; interrupted during epoch 8	150 (recorded)
Batch size	16	16 (recorded)
Image size	640	1024 × 1024 (recorded)
Optimiser / learning rate	optimizer = auto → AdamW, lr ≈ 0.0009 (lr0 = 0.01 ignored)	Not recorded
Dataset configuration	data.yaml; classes: cut, good, husk, immature, parchment, partial-black, shell	Same class list
Checkpoint	runs/segment/train/weights/best.pt, last.pt	Copy of v8_75.pt
Export	—	ONNX export (nominal 1024 × 1024) recorded

At inference time the trained checkpoint is loaded and applied to an image. For each detected bean the result contains a bounding box, a confidence score, a class and a mask. The deployment notebook uses a confidence threshold of 0.5; other notebook cells used 0.25 and 0.8 for exploratory predictions. A confidence threshold controls the trade-off between missed beans and false detections. On the sample image of the deployment notebook the model, run at 1024 × 1024, detected 13 beans.

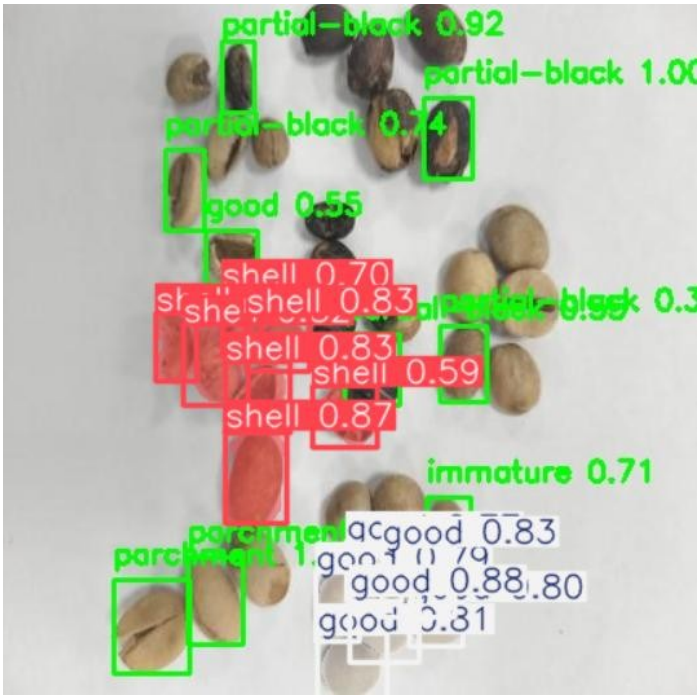


Figure 4.1: *YOLOv8n-seg output on a sample image, showing the class label and confidence predicted by the segmentation model itself*

4.4 Bean Extraction

Bean extraction turns segmentation results into individual bean images. In the deployment notebook it is implemented as the following sequence for each detected instance:

1. The instance mask is resized to the size of the original image with nearest-neighbour interpolation and thresholded at 0.5.
2. All pixels outside the mask are set to zero (black), so the bean is shown on a plain black background.
3. The masked image is cropped to the bounding box of the instance.
4. The crop is resized to 224 × 224 pixels, converted from BGR to RGB and passed to the DINOv2 image processor.

Several risks affect this step. An inaccurate mask may remove part of the bean, including the defect that defines its class, or include background that misleads the classifier. When beans overlap or touch, a mask may merge two beans or cut one bean in two. Very small or partial masks may produce crops with too few pixels. Resizing every crop to a square changes the aspect ratio of elongated beans. For these reasons it is recommended to inspect samples of extracted beans for each class. The bean images from which the training features were computed were stored in class folders (the Coffee_DINO dataset); the procedure that generated them is not part of the supplied notebooks, so the equivalence of training-time and inference-time crops is a design intention that has not been verified.

4.5 DINOv2 Feature Extraction

Feature extraction loads the image processor and the model for facebook/dinov2-base through Hugging Face Transformers. Each bean image is processed by the image processor, which resizes

and normalises it, and the resulting tensor is passed through the model in evaluation mode without gradient computation, so the encoder remains frozen. The embedding of a bean is the output vector at the first (classification, CLS) token position of the last hidden state, which has 768 dimensions, and one embedding corresponds to one bean image. In the feature-extraction notebook the images were processed in batches of 32 on a GPU, with the data loader keeping a fixed order (no shuffling) so that each embedding stays aligned with its label.

The embeddings for the training, validation and test partitions were saved as compressed NumPy arrays together with the labels. The correspondence between embeddings and labels is critical: the i-th embedding must belong to the i-th label. This is ensured by processing images in a fixed order, saving the label array in the same pass, and checking that the arrays have matching lengths.

Table 4.3: DINOv2 feature extraction configuration

Item	Value
Model	facebook/dinov2-base, used as a frozen feature extractor (no fine-tuning)
Embedding dimension	768
Embedding definition	CLS-token vector: outputs.last_hidden_state[:, 0, :]
Image processor	Hugging Face AutoImageProcessor for facebook/dinov2-base (settings given by its preprocessor_config.json)
Feature-extraction batch size	32 (batched on a GPU); one bean per forward pass in the deployment notebook
Saved files	train_data.npz, valid_data.npz, test_data.npz (arrays “features” and “labels”)
Feature array shapes	Train (4368, 768); validation (182, 768); test (215, 768) — float32 features
Label array shapes	Train (4368,); validation (182,); test (215,)

4.6 CoffeeMLP Classifier

CoffeeMLP takes a 768-dimensional embedding as input and has the architecture 768 → 512 → 256 → 7 with batch normalisation, ReLU and dropout in the hidden layers (Section 3.5.3). It is trained on the saved training features and labels, validated on the validation features after every epoch, and finally evaluated on the test features. For prediction, the seven output scores are passed through an argmax, and the resulting index is mapped to a class label using the mapping of the class-mapping table. In the deployment notebook the confidence value reported for a bean is the softmax value of the predicted class.

Table 4.4: CoffeeMLP training configuration

Item	Value
Loss function	Cross-entropy loss (nn.CrossEntropyLoss)
Optimiser	Adam, learning rate 0.001; no weight decay and no learning-rate scheduler
Batch size	64 (training data shuffled); 64 for validation and test (not shuffled)
Number of epochs	30
Dropout probability	0.30 (hidden block 1); 0.20 (hidden block 2)
Class weighting / sampling	None used
Checkpoint-selection rule	The model is saved whenever the validation accuracy is strictly higher than the best value so far

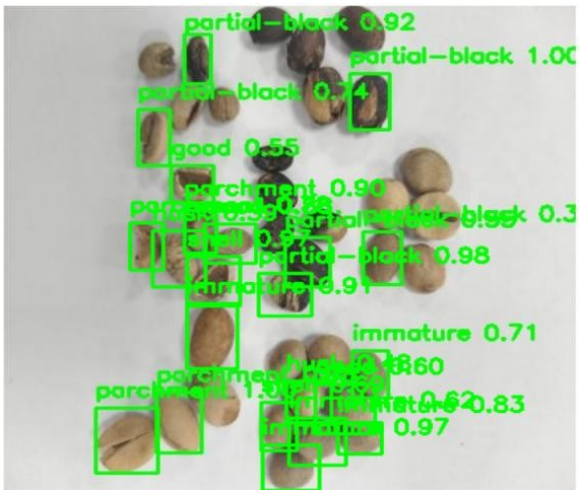
Item	Value
Saved classifier file	mlp_classifier.pth (state dictionary of CoffeeMLP)
Random seed	No random seed was set in the notebook

The validation accuracy over the 30 epochs ranged between 0.9011 and 0.9451. The highest value, 0.9451 (172 of 182 validation samples), was first reached in epoch 13 and not exceeded afterwards, so the saved checkpoint corresponds to that epoch. Because this checkpoint was chosen on the validation partition, the validation accuracy is an optimistic estimate; the unbiased estimate is the test result of Chapter 5.

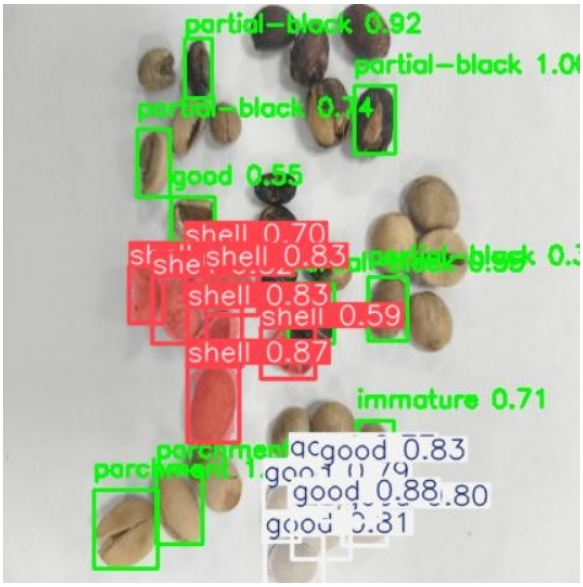
4.7 Integration

Integration connects the stages so that an image can be processed end to end. Four aspects need attention. First, checkpoint compatibility: the segmentation checkpoint, the DINOv2 model and the CoffeeMLP weights must correspond to the versions used during training, and the classifier's input size must equal the embedding size (768). Second, consistent preprocessing: bean images at inference must be prepared in the same way as during feature extraction for training. Third, error handling: the pipeline should handle images with no detected bean, unreadable files and missing checkpoints, and report them instead of returning misleading labels; the notebooks do not yet implement this. Fourth, output generation: results list, for each bean, its location, the predicted class and the softmax confidence.

The deployment notebook (4_Deployment_Pipeline.ipynb) combines the stages. It loads the segmentation checkpoint, the DINOv2 model and the CoffeeMLP weights, predicts masks for an uploaded image, extracts each bean, computes its embedding and class, draws a bounding box and a "class confidence" label on a copy of the image, and saves the annotated image. In this notebook the class shown for a bean is the CoffeeMLP prediction; the class predicted by the segmentation model is not used for the final label. Because the segmentation model was trained with the same seven classes, the two predictions can be compared; where they disagree, the segmentation model's label should not be mistaken for the final classification.



(a) CoffeeMLP labels (green boxes)



(b) YOLOv8n-seg labels

Figure 4.2: *Per-bean labels for one input image: (a) pipeline output with CoffeeMLP labels; (b) class labels predicted directly by YOLOv8n-seg*

The two outputs in the figure above differ for several beans. No ground-truth labels are available for this image, so the figure does not show which labelling is correct; it is reported as a qualitative observation and as motivation for the end-to-end evaluation proposed in Chapter 6.

4.8 Deployment Investigation

The project included an experiment in exporting a trained model to ONNX and an exploration of deployment on an Arduino UNO Q platform [17]. This work is presented as an investigation. A deployment effort involves several distinct steps, and success at one does not imply success at the next: exporting a model produces a file; loading it in a runtime shows that the runtime supports its operations; running inference shows that outputs can be produced; deploying the whole pipeline requires all stages, including preprocessing and feature extraction, to run on the device; and measuring performance requires timing and memory records. The next table records what is established.

Table 4.5: Deployment investigation status

Step	Status	Evidence
Export trained segmentation model to ONNX	Recorded as performed	ONNX files for the 640-pixel and 1024-pixel checkpoints (best_1.onnx, v8_75.onnx) are recorded in the project.
Load model in an ONNX-capable runtime	Not verified	Runtime-loading evidence not supplied.
Run inference on the target device	Not performed	Arduino UNO Q not yet available.
Run the complete pipeline on the device	Not established	Complete on-device execution not demonstrated.
Measure latency and memory on the device	Not established	No on-device measurements available.

This report makes no claim that the pipeline was successfully deployed on any edge device. The DINOv2 encoder is considerably larger than YOLOv8n, so running the whole pipeline on a small device may need techniques such as quantisation or a smaller feature extractor, which are listed as future work. Replacing the feature extractor would require the classifier to be retrained or revalidated, because CoffeeMLP was trained on DINOv2-base embeddings.

4.9 User Interface

The project is operated through Jupyter-style notebooks in Google Colab: the user runs cells to load data, train, extract features, classify and view results. Inference results are displayed in the notebook output, and the annotated image is also saved to a file. No graphical user interface, web application or dashboard is part of the results reported here. The next figure shows the notebook-based inference workflow.

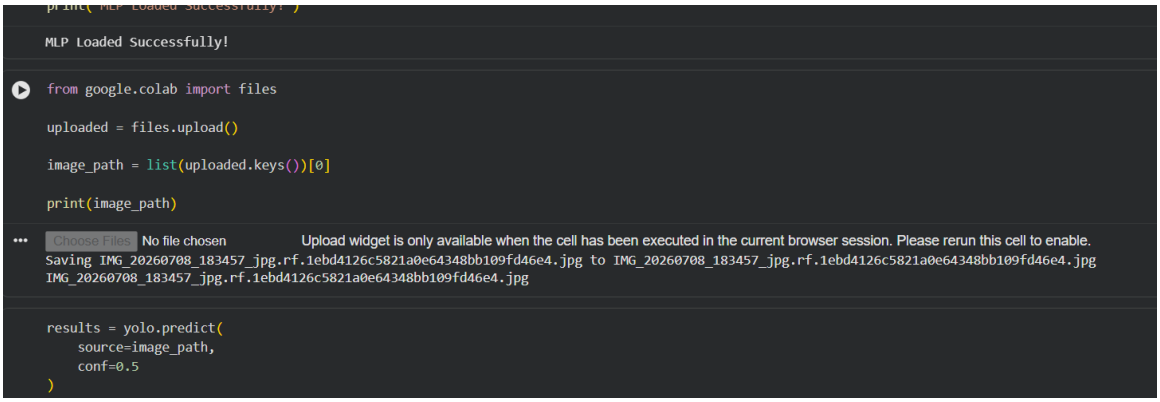


Figure 4.3: *Notebook inference workflow (a): loading the models and supplying an input image*

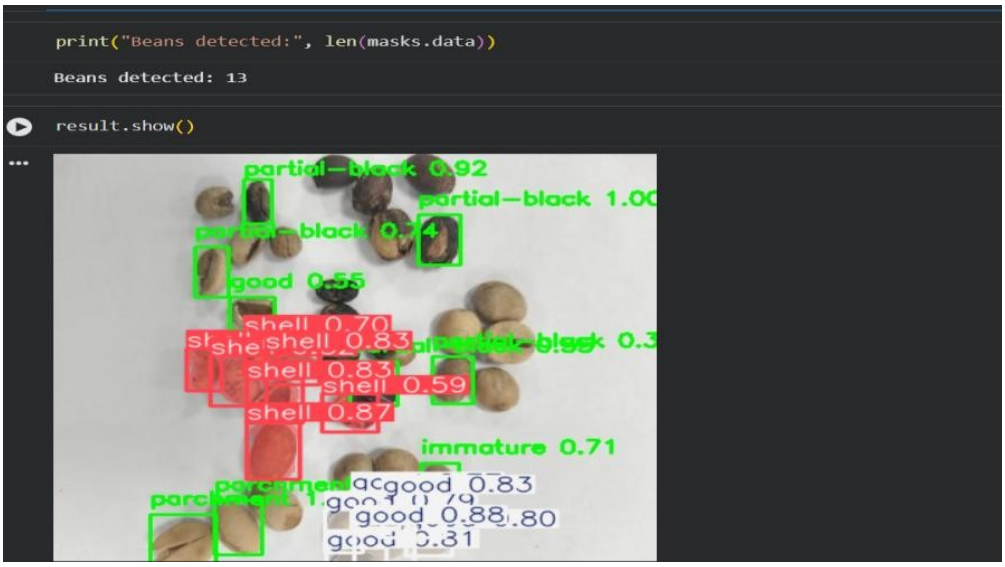


Figure 4.4: *Notebook inference workflow (b): segmentation of the input image (13 beans detected)*

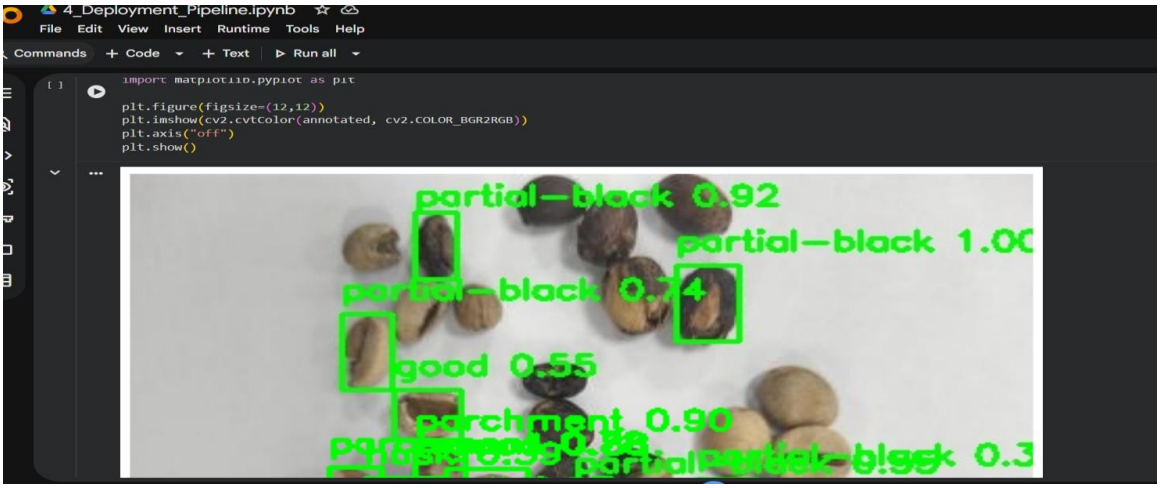


Figure 4.5: *Notebook inference workflow (c): annotated result produced by the deployment notebook*

CHAPTER 5: SOFTWARE TESTING AND RESULTS

This chapter distinguishes three kinds of statement: results that were measured, results that were recorded in earlier experiments and still have to be confirmed, and tests that are specified but have not yet been run. Each result is reported together with the data and checkpoint it refers to.

5.1 Testing Methodology

Testing combines software testing with machine-learning evaluation. The next table summarises the levels used.

Table 5.1: Testing levels applied to CoffeeBeanAI

Level	Purpose	Examples
Unit testing	Check individual functions in isolation.	Class-index mapping; bean cropping within bounds; output shape of CoffeeMLP.
Data validation	Check the dataset before training.	Matching image/label files; class indices in 0–6; no image in more than one partition; feature and label arrays of equal length.
Model evaluation	Measure the quality of each model on held-out data.	Segmentation: precision, recall, mAP. Classification: accuracy, precision, recall, F1, confusion matrix.
Integration testing	Check that stages work together.	Segmentation output accepted by extraction; embedding of size 768 accepted by CoffeeMLP.
End-to-end testing	Run the complete pipeline on whole images.	Image in, class label for every bean out; behaviour with no detected bean.

5.2 Segmentation Evaluation

The segmentation model is evaluated with four metrics. For this evaluation, a predicted mask or box is matched to a ground-truth object using the intersection over union (IoU):

$$IoU = |A \cap B| / |A \cup B| \tag{5.1}$$

where A is the predicted region and B the ground-truth region. A prediction is a true positive (TP) if its IoU exceeds a threshold and its class is correct; otherwise it is a false positive (FP); a ground-truth object with no matching prediction is a false negative (FN). Precision and recall are then

$$Precision = TP / (TP + FP), \quad Recall = TP / (TP + FN) \tag{5.2}$$

Precision measures how many of the predicted beans are correct, and recall measures how many of the real beans were found. Average precision (AP) is the area under the precision–recall curve for a class [14], and mAP is the mean of AP over classes. mAP@0.50 uses an IoU threshold of 0.50; mAP@0.50:0.95 averages the result over IoU thresholds from 0.50 to 0.95 in steps of 0.05 and therefore rewards accurate boundaries [2]. The Ultralytics library reports these metrics for both boxes and masks.

5.2.1 Previously Reported Segmentation Results

Earlier experiments recorded the values below. They are previously reported results of the segmentation experiment; they are not results of the classifier evaluation of Section 5.3 and must not be combined with it. They should be confirmed against the original segmentation evaluation logs, the checkpoint and the dataset split before being described as a fully reproduced benchmark, and it should also be confirmed whether they refer to box or mask metrics. The segmentation training notebook supplied for this report was interrupted during epoch 8 and does not display final evaluation values, so these figures cannot be traced to that run.

Table 5.2: Previously reported YOLOv8n segmentation metrics

Metric	Previously reported value	Status
Precision	97.59% (0.9759)	Recorded; to be confirmed
Recall	89.27% (0.8927)	Recorded; to be confirmed
mAP@50	94.87% (0.9487)	Recorded; to be confirmed
mAP@50–95	92.62% (0.9262)	Recorded; to be confirmed

These values describe the segmentation model only. They do not state how accurately beans are classified, and they must not be used as a measure of the final system’s performance. If confirmed, the recall being lower than the precision would mean that, under the recorded evaluation, some beans were missed more often than beans were falsely detected.

5.3 Classification Evaluation

The DINOv2 + CoffeeMLP classifier is evaluated separately on the held-out test features and labels. For a class k, precision and recall are defined as above, with TP, FP and FN counted for that class, and the F1-score is their harmonic mean:

$$F1 = 2 \times Precision \times Recall / (Precision + Recall) \tag{5.3}$$

Accuracy is the fraction of correctly classified beans. Macro-averaged precision, recall and F1 are the unweighted means of the per-class values, so every class counts equally regardless of its size, which is important if the classes are unbalanced. Weighted averages weight each class by its number of test samples and therefore follow the larger classes. The confusion matrix counts, for every pair of classes, how often a bean of one actual class was predicted as another: the diagonal entries are correct classifications and the off-diagonal entries are errors.

5.3.1 Evaluation Setup

Table 5.3: Setup of the classifier evaluation

Item	Value
Evaluated checkpoint	Saved CoffeeMLP checkpoint (mlp_classifier.pth)
Evaluation script	src/evaluate_mlp.py (the same metrics were obtained in the classifier notebook)
Test features	Matrix of shape (215, 768), float32, precomputed with DINOv2 (processed test features of the repository)
Test labels	Array of shape (215,), int64
Device	CPU (recorded evaluation device)

Item	Value
Input to the classifier	Stored DINOv2 embeddings; the evaluation does not run segmentation, bean extraction or DINOv2 on images

5.3.2 Overall and Per-Class Results

Table 5.4: Overall classification metrics on the test set

Metric	Value
Accuracy	0.8977 (approximately 89.77%)
Macro precision	0.89
Macro recall	0.90
Macro F1-score	0.89
Weighted precision	0.90
Weighted recall	0.90
Weighted F1-score	0.90
Test support	215

Table 5.5: Per-class classification metrics on the test set

Class	Precision	Recall	F1-score	Support
cut	0.85	0.96	0.90	23
good	0.94	1.00	0.97	58
husk	0.98	0.80	0.88	54
immature	0.90	0.82	0.86	11
parchment	0.92	1.00	0.96	11
partial-black	0.83	0.81	0.82	31
shell	0.81	0.93	0.86	27
Macro average	**0.89**	**0.90**	**0.89**	**215**
Weighted average	**0.90**	**0.90**	**0.90**	**215**

The report values are rounded to two decimal places. The accuracy of 0.8977 corresponds to 193 correctly classified samples out of 215. Assuming that the test samples are independent, an approximate 95% Wilson confidence interval for this accuracy is 0.85 to 0.93. This interval is a simple indication of the uncertainty caused by the size of the test set; no significance test or comparison between models has been carried out.

5.4 Confusion Matrices

The classifier evaluation produces a raw confusion matrix and a normalised confusion matrix, stored in the project repository under outputs/confusion_matrix/ (confusion_matrix.png, confusion_matrix_normalized.png and confusion_matrix.csv). In a raw confusion matrix, each row corresponds to an actual class and each column to a predicted class; the diagonal entries count

correct classifications and every off-diagonal entry counts a particular kind of error. In a row-normalised matrix, each row is divided by the number of samples of that actual class, so that the diagonal entries equal the per-class recall. The class order of the matrices is the order of the class mapping (0 cut to 6 shell), the same as in the per-class table above. These matrix files were not available when this report was prepared; the report therefore does not reproduce them or state individual confusion counts. Statements about which classes are confused with which can only be made from the matrices, and the analysis below is limited to what the per-class metrics imply.

5.5 Three Levels of Evaluation

The metrics in this chapter come from different tasks and must be kept apart. The next table summarises what has and has not been evaluated.

Table 5.6: Levels of evaluation and their status

Level	What is measured	Status
1. Instance segmentation	Whether beans are found and outlined correctly (precision, recall, mAP on boxes and masks).	Previously reported values (Table 5.2); to be confirmed against logs.
2. Feature-based classification	Whether CoffeeMLP assigns the correct class to a stored DINOv2 embedding of a bean.	Measured: accuracy 0.8977 on 215 test samples.
3. Full pipeline	Whether an uploaded image is turned into correct per-bean labels, including the effect of segmentation errors and of the crop preparation.	Not yet evaluated; future work.

The classifier accuracy of 89.77% is therefore not the accuracy of the complete YOLOv8 → bean extraction → DINOv2 → MLP system, and it must not be merged with the segmentation metrics into a single figure.

5.6 Test Case Specifications

The next table specifies tests for the main components and states what evidence exists for each. A status of *Pending* means that the test has been specified but no test evidence has been produced.

Table 5.7: Test case specifications and status

ID	Component	Condition	Expected result	Status / evidence
TC-01	Image loading	Valid image file	RGB array with 3 channels	Passed (notebook upload cell)
TC-02	Image loading	Missing or unreadable path	Clear error; no crash	Pending
TC-03	Segmentation	Image with several beans	One mask per detected bean	Passed on one image (13 detections, 13 masks)
TC-04	Segmentation	Image with no beans	Empty result handled correctly	Pending
TC-05	Mask validity	Predicted mask	Matching dimensions; non-empty area	Pending
TC-06	Annotation	Label file with invalid class or malformed line	Error reported	Pending
TC-07	Class mapping	Indices 0–6	Identical in all places	Passed (inspection of the four notebooks)

ID	Component	Condition	Expected result	Status / evidence
TC-08	Bean extraction	Mask and bounding box	Non-empty crop inside image bounds	Passed visually on one bean; no systematic test
TC-09	Preprocessing	Bean image	Tensor of expected shape	Pending
TC-10	Feature dimension	Batch of bean images	Shape (N, 768)	Passed (shapes of the saved arrays; single-bean feature of shape (1, 768))
TC-11	Feature–label alignment	Saved feature and label arrays	Equal lengths; labels match images	Lengths equal; spot-check not recorded
TC-12	Checkpoint loading	Missing checkpoint path	Informative error	Pending
TC-13	Classifier output	Embedding batch	Shape (N, 7); indices in 0–6	Passed (all state-dictionary keys matched; 215 predictions produced)
TC-14	Evaluation integrity	Train, validation and test files	No overlap	Pending
TC-15	End-to-end	Whole image	A label for every detected bean	Executed on one image; not evaluated against ground truth
TC-16	ONNX export	Exported model	Outputs agree within tolerance	Pending

5.7 Error Analysis

A complete error analysis requires the confusion matrix and inspection of misclassified examples. The per-class metrics already allow some careful statements. With 193 of 215 samples correct, there are 22 classification errors in total. The statements below are inferred from rounded metrics and should be checked against the confusion matrix.

- **good** has a recall of 1.00 (all 58 samples found) but a precision of 0.94, which implies that a few beans of other classes were predicted as *good*.
- **parchment** has a recall of 1.00 and **immature** a recall of 0.82, but each has only 11 test samples, so one misclassified sample changes the recall by about 9 percentage points and these values should be read with caution.
- **husk** has the lowest recall (0.80), which implies that roughly 11 of its 54 test samples were assigned to other classes, whereas its precision of 0.98 means that almost every bean predicted as *husk* was a husk bean.
- **partial-black** has the lowest F1-score (0.82), with precision 0.83 and recall 0.81, so it is affected by errors in both directions.
- **shell** has a recall of 0.93 but the lowest precision (0.81), which implies that several beans of other classes were predicted as *shell*.

Which classes are confused with which cannot be stated without the confusion matrix. The reasons for the errors are not known; possible factors such as visual similarity between classes, class imbalance, limited training data, lighting and background differences, bean orientation and annotation ambiguity should be examined rather than assumed. Segmentation errors can also propagate into classification errors in the full pipeline: a missed bean is never classified; a mask that

cuts off part of a bean may remove the defect that defines its class; and a mask that merges two beans can produce an embedding that represents neither. These effects are not captured by the feature-based test of Section 5.3, because the test features are already prepared. To separate them, classification can be evaluated both on beans cropped from ground-truth masks and on beans cropped from predicted masks.

5.8 Performance and Deployment Testing

A systematic benchmark of inference time, throughput and memory use has not been carried out. The next table lists the incidental timings that appear in the logs of the notebooks. They are single runs on shared notebook hardware, include data-loading overheads where stated, and must not be read as benchmarks or as indicators of performance on an edge device.

Table 5.8: Incidental timings recorded in notebook logs (not a benchmark)

Measurement	Recorded value	Conditions
DINOv2 feature extraction, training partition	10 min 31 s for 4,368 images	Colab T4 GPU, batch size 32, including image loading from Google Drive
DINOv2 feature extraction, validation / test	33 s for 182 images / 41 s for 215 images	Same conditions
YOLOv8n-seg prediction, 1024 × 1024 image	26.6 ms preprocess, 23.7 ms inference, 5.1 ms postprocess	Single image, deployment notebook, CPU runtime
Segmentation + DINOv2 + CoffeeMLP end-to-end time per image	Not measured	—
Peak memory	Not measured	—
Edge device (Arduino UNO Q)	Not tested	—

No conclusions about real-time performance or edge-device efficiency are drawn at this stage.

5.9 Results Discussion

The classification results provide the main quantitative evidence of the project's central claim that frozen DINOv2 features and a small MLP can separate the seven bean categories. On the 215-sample test set the saved classifier reached an accuracy of 0.8977 and a macro F1-score of 0.89. The *good* and *parchment* classes were recognised with a recall of 1.00, while *husk* (recall 0.80) and *partial-black* (F1-score 0.82) were the weakest in recall and F1 respectively, and *shell* had the lowest precision (0.81). The validation accuracy of the selected checkpoint (0.9451) is higher than the test accuracy; with validation and test sets of 182 and 215 samples and a checkpoint selected on the validation set, a difference of this size is not unexpected, but its cause has not been investigated.

These results should be interpreted with several points in mind. The overall accuracy should not be read in isolation, because the classes are unevenly represented in the test set (11 to 58 samples), and small classes give unstable estimates. The test set contains images from the same dataset as the training data, so the results say little about other lighting conditions, cameras or bean batches. The evaluation uses stored features and does not include the segmentation stage. The previously reported segmentation metrics, if confirmed, would suggest that the segmentation model finds and outlines most beans of its evaluation data, but they do not show how well the beans are classified in the

complete pipeline. Further validation would be needed before any claim about practical grading: end-to-end evaluation on whole images, images from other settings, comparison with expert inspectors, and measured throughput on the intended hardware.

CHAPTER 6: CONCLUSION AND FUTURE WORK

6.1 Current Status and Conclusion

CoffeeBeanAI addresses the problem of assessing coffee-bean quality from images. The project developed a modular pipeline in which a YOLOv8n instance-segmentation model identifies individual beans and produces masks, the beans are extracted and preprocessed, a pretrained DINOv2 encoder (facebook/dinov2-base) converts each bean image into a 768-dimensional embedding, and a CoffeeMLP classifier with the architecture 768 → 512 → 256 → 7 assigns it to one of seven categories: cut, good, husk, immature, parchment, partial-black and shell.

The role of each component is distinct. Segmentation localises beans and separates them from the background and from each other. DINOv2 supplies general-purpose visual features so that a small classifier can be trained on a limited labelled dataset. CoffeeMLP learns the mapping from these features to the project's classes. Evaluating the stages separately makes it possible to attribute errors to a stage.

On the held-out test set of 215 precomputed feature vectors, the saved CoffeeMLP checkpoint achieved an accuracy of 0.8977 (approximately 89.77%) and a macro-averaged F1-score of 0.89. This is a result for the classification stage on stored features and not for the complete pipeline. The segmentation results recorded during experimentation are reported in Chapter 5 as previously reported values that remain to be confirmed against their original logs. An ONNX export and an exploration of edge deployment were carried out as investigations; successful end-to-end deployment has not been established. The conclusions of this report are limited accordingly.

6.2 Limitations

The limitations are listed below in two groups: those that follow directly from the project material, and general methodological considerations that apply to this kind of study.

Table 6.1: Known limitations and proposed improvements

Limitation	Proposed improvement
Confirmed from the project material	
The test partition is small and uneven; *immature* and *parchment* have only 11 test samples each.	Enlarge the test set; report confidence intervals; use cross-validation or repeated splits.
The classifier was evaluated on precomputed features; no end-to-end evaluation exists.	Evaluate the complete image-to-label pipeline on the test images.
The segmentation metrics cannot be traced to a logged run, and the training configuration of v8_75.pt (100 vs 150 epochs; 640 vs 1024 pixels) is not documented.	Re-run or re-validate the segmentation model and record the log, checkpoint and split.
The recipe that generated the training bean images is not documented, so training-time and inference-time crops may differ.	Document the crop recipe, or regenerate the training features with the inference preprocessing and re-evaluate.
On one qualitative example the labels of the segmentation model and of CoffeeMLP differ markedly (Figure 4.2).	Evaluate both label sources on annotated images and define which one is final.
Dataset source and licence are not documented; per-class training and validation counts could not be reconciled with the split totals.	Document dataset provenance and licence; regenerate the per-class counts from the label files.

Limitation	Proposed improvement
Latency and memory use are unmeasured; edge deployment is unverified.	Measure inference time and memory; try quantisation; validate on the target hardware.
Methodological considerations	
Annotation consistency between similar classes may limit accuracy.	Review annotations; define clear labelling guidelines; measure inter-annotator agreement.
Segmentation errors propagate to classification.	Compare classification on ground-truth and predicted masks; improve segmentation.
Results are limited to the imaging conditions of the dataset.	Test under different lighting, backgrounds and cameras.
The frozen DINOv2 encoder is not adapted to coffee beans.	Compare with fine-tuning or other feature extractors.
Softmax scores are not calibrated probabilities.	Assess calibration before using confidence thresholds.

The system has not been certified for industrial grading and has not been validated under factory conditions.

6.3 Future Work

The following directions are proposed. They are plans, not completed work.

- Evaluate the complete image-to-prediction pipeline and report end-to-end results separately from the stage-wise results.
- Repeat the segmentation evaluation using recorded checkpoints and configurations, and archive the logs.
- Increase dataset diversity, review annotation quality and, where justified, balance the classes.
- Improve fine-grained classification among the weaker classes (husk, partial-black, shell) after examining the confusion matrix.
- Perform systematic hyper-parameter tuning of the classifier and compare alternative classifiers and feature extractors (for example other DINOv2 sizes), retraining the classifier whenever the feature extractor changes.
- Measure inference latency, throughput and memory on the intended platforms, and investigate quantisation and other optimised inference methods.
- Validate the complete pipeline on edge hardware, using the ONNX investigation as a starting point.
- Test robustness under different lighting and imaging conditions.
- Investigate real-time camera integration only if it becomes part of the project’s scope.

CHAPTER 7: SUMMARY

7.1 Summary

CoffeeBeanAI is a research project on image-based coffee-bean quality assessment. It was motivated by the limits of manual inspection—limited throughput, subjectivity and poor repeatability—and aims to study whether a modular computer-vision pipeline can classify individual beans into seven visible categories: cut, good, husk, immature, parchment, partial-black and shell.

The architecture has three learned components. YOLOv8n-seg segments each bean; the masks are used to extract individual bean images; a frozen DINOv2 encoder turns each image into a 768-dimensional embedding; and CoffeeMLP (768 → 512 → 256 → 7) predicts the class. The dataset was organised into training, validation and test partitions with a fixed class mapping, and the implementation used Python, PyTorch, Ultralytics and Hugging Face Transformers in Google Colab.

Evaluation keeps the two parts of the pipeline apart. The classifier reached an accuracy of about 89.77% (macro F1-score 0.89) on 215 test samples of precomputed features; segmentation metrics recorded in earlier experiments are reported separately and remain to be confirmed. ONNX export and edge deployment were explored but are not claimed as a completed deployment.

The main limitations are the small and uneven test classes, the absence of an end-to-end evaluation, the undocumented training-crop recipe and segmentation training log, the dependence of classification on mask quality, limited testing under varied conditions and the absence of measured deployment performance. Next steps are to evaluate the complete pipeline, confirm the segmentation results, broaden the data, compare alternative models, measure latency and test on edge hardware.

REFERENCES

[1] G. Jocher, A. Chaurasia and J. Qiu, “Ultralytics YOLOv8,” 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[2] Ultralytics, “Ultralytics YOLO Docs.” [Online]. Available: https://docs.ultralytics.com/

[3] J. Redmon, S. Divvala, R. Girshick and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2016, pp. 779–788.

[4] K. He, G. Gkioxari, P. Dollár and R. Girshick, “Mask R-CNN,” in Proc. IEEE Int. Conf. Computer Vision (ICCV), 2017, pp. 2961–2969.

[5] M. Oquab et al., “DINOv2: Learning Robust Visual Features without Supervision,” Transactions on Machine Learning Research, 2024; arXiv:2304.07193.

[6] A. Dosovitskiy et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in Proc. Int. Conf. Learning Representations (ICLR), 2021; arXiv:2010.11929.

[7] Hugging Face, “facebook/dinov2-base” model card. [Online]. Available: https://huggingface.co/facebook/dinov2-base

[8] T. Wolf et al., “Transformers: State-of-the-Art Natural Language Processing,” in Proc. Conf. Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP), 2020, pp. 38–45.

[9] A. Paszke et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” in Advances in Neural Information Processing Systems 32 (NeurIPS), 2019, pp. 8024–8035.

[10] PyTorch, “PyTorch Documentation.” [Online]. Available: https://pytorch.org/docs/stable/

[11] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in Proc. Int. Conf. Machine Learning (ICML), 2015, pp. 448–456.

[12] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever and R. Salakhutdinov, “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” Journal of Machine Learning Research, vol. 15, pp. 1929–1958, 2014.

[13] V. Nair and G. E. Hinton, “Rectified Linear Units Improve Restricted Boltzmann Machines,” in Proc. Int. Conf. Machine Learning (ICML), 2010, pp. 807–814.

[14] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn and A. Zisserman, “The PASCAL Visual Object Classes (VOC) Challenge,” International Journal of Computer Vision, vol. 88, no. 2, pp. 303–338, 2010.

[15] ONNX Project, “ONNX: Open Neural Network Exchange.” [Online]. Available: https://onnx.ai/

[16] C. R. Harris et al., “Array programming with NumPy,” Nature, vol. 585, pp. 357–362, 2020.

[17] Arduino, “Arduino UNO Q (SKU ABX00162 / ABX00173) User Manual,” modified 13 Oct. 2025. [Online]. Available: https://docs.arduino.cc/

APPENDIX A: LIST OF ABBREVIATIONS

Table A.1: List of abbreviations

Abbreviation	Full form
AI	Artificial Intelligence
AIML	Artificial Intelligence and Machine Learning
AP	Average Precision
BN	Batch Normalization
CEA	Computer Engineering and Applications
CLS	Classification token of a vision transformer
CNN	Convolutional Neural Network
CSV / NPY / NPZ	Comma-Separated Values / NumPy array file formats
DINOv2	Self-DIstillation with NO labels, version 2
FN / FP / TP	False Negative / False Positive / True Positive
GLAU	GLA University
GPU	Graphics Processing Unit
IoU	Intersection over Union
mAP	mean Average Precision
ML	Machine Learning
MLP	Multilayer Perceptron
ONNX	Open Neural Network Exchange
ReLU	Rectified Linear Unit
RGB	Red, Green, Blue
UML	Unified Modeling Language
ViT	Vision Transformer
YOLO	You Only Look Once

APPENDIX B: CLASS MAPPING AND SAMPLE INFERENCE WORKFLOW

B.1 Class Mapping

Table B.1: Class mapping used throughout the project

Class ID	Class label
0	cut
1	good
2	husk
3	immature
4	parchment
5	partial-black
6	shell

This mapping is used in the dataset configuration, the saved label arrays, the classifier and the inference code.

B.2 Selected Configuration

- Segmentation: YOLOv8n-seg initialised from yolov8n-seg.pt; checkpoint v8_75.pt used for inference at 1024 × 1024 with a confidence threshold of 0.5; training configuration of the 640-pixel notebook run: 100 epochs requested, batch size 16.
- Feature extractor: facebook/dinov2-base, frozen; CLS-token embedding of dimension 768.
- Classifier: CoffeeMLP, 768 → 512 → 256 → 7; cross-entropy loss; Adam, learning rate 0.001; 30 epochs; batch size 64.
- Dataset: newMaskedDataset (YOLOv8 segmentation format); features: DINO_Features (train_data.npz, valid_data.npz, test_data.npz).

B.3 Condensed Inference Workflow

The following listing is a condensed outline of the deployment notebook, not a verbatim excerpt; it shows the order of operations.

```
CLASSES = ["cut","good","husk","immature","parchment","partial-black","shell"]
result = yolo.predict(source=image_path, conf=0.5)[0]       # masks, boxes
for i in range(len(result.masks.data)):
    mask = resize(result.masks.data[i], image_size) > 0.5   # nearest neighbour
    bean = image.copy(); bean[mask == 0] = 0                # black background
    crop = resize(bean[y1:y2, x1:x2], (224, 224))           # bounding-box crop
    rgb  = bgr_to_rgb(crop)
    feat = dinov2(**processor(images=rgb)).last_hidden_state[:, 0, :]   # 768-d
    prob = softmax(mlp(feat), dim=1)
    label, conf = CLASSES[argmax(prob)], max(prob)
```